# The Closed Record

### Google's own technical report, Google's own code, Google's own issue boards and Google's own patent counsel: the Compressed Memory Manifold and Intelligent Intelligence Class II in Google's stack

**Green Recursive Utility Service LLC** · Weatherford, Texas · October 16, 2026
**Nicholas Wray Cordova**, Principal and Inventor of Record

USPTO 19/700,381 (CMM, priority May 14, 2026) · USPTO 19/710,080 (II-II, priority May 28, 2026) · PUCT Control No. 59976-1 · FERC Accession 20260716-5137 · Parker County OPR Doc. 202618373

---

> [!IMPORTANT]
> **Don't take our word for any of it. Run it.**
>
> Google's code, from Google's own public repository:
>
> ```
> GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/google-ai-edge/LiteRT-LM.git && cd LiteRT-LM
> grep -rn "row_offset\|chunk_size_bytes\|multiple of 64\|UnpackInt4Row\|PrefetchToLocalCache" runtime/executor/
> grep -n "checkpoint_map_\|SetCurrentStep" runtime/core/session_advanced.cc
> ```
>
> GRUS's block fingerprint, from the store running since November 2025:
>
> ```
> python3 -c "import
> base64,hashlib;b=base64.b64decode('AAAABiF36zkaG8yHy11aBWGfx9F4/dbrlyCi09A9hxVjYWwgZmxvdywgb2JqZWN0IHRyYWNraW5
> nCjYuIEltYWdlIHN0aXRjaGluZyBmb3IgcGFub3JhbWFzCjcuIEZh');print(b[4:32]==hashlib.sha256(b[32:]).digest()[:28])"
> ```
>
> Output: `True`

---

## Timeline

```
timeline
    title GRUS first, Google after
    2025 : May 8 GRUS Lockdown License on GitHub
         : May 12 Class I.I. repository created
         : Sep 5 Digital Physics Charter
         : Sep 15 Class II + CMM announced
         : Nov CMM block library running on bare metal
    Feb 2026 : Feb 17 The only proposal, to nine Alphabet channels
    May 2026 : May 8 Hardware-counter measurements
             : May 14 CMM filed and announced
             : May 28 II-II filed
    Jun 2026 : Jun 5 Google releases Gemma 4 + LiteRT-LM
             : Jun 6 Notice within 24 hours
             : Jun 8 and 11 Invoice and service agreement acknowledged in Google procurement
    Jul 2026 : Jul 2 Gemma 4 Technical Report
             : Jul 10 and 16 Tariff filed at PUCT and FERC
             : Jul 24 Google patent counsel declines "your proposal"
    Aug-Oct 2026 : Aug 6 FERC window closes without protest
                 : Aug 12 Pixel 11
                 : Oct 6 EmbeddingGemma 2
```

---

## Contents


- The record in one page
- 1. The method, as GRUS defined it before June 5
- 2. GRUS built it first: the 2025 record
- 3. The running system, its fingerprint and its VIN
- 4. The only proposal
- 5. The filings and the public announcement
- 6. June 5, 2026: the release
- 7. Google's own technical report
- 8. Google's own code
- 9. Google's own issue boards
- 10. Google's own announcements
- 11. Google's own counsel
- 12. The notice chain
- 13. Preservation, and what happened after notice
- 14. The physics
- 15. Every route ends at the bill
- 16. Answered before it is asked
- 17. It doesn't stop at GRUS
- 18. Nothing left to argue
- Appendix A. Identifiers of record
- Appendix B. The census
- Appendix C. Sources


---

## The record in one page

Every layer below is independent. Each one stands without the others, and each one is checkable by anyone, from the source named beside it.

| # | Layer | What it shows | Where to check it |
|---|---|---|---|
| 1 | **GRUS's 2025 copyrights** | The CMM and Class I.I. work, its block lanes and its no-reuse license, fixed in public GitHub repositories in May 2025 | GitHub's own server timestamps (Class I.I., chloe-runtime) |
| 2 | **The running system** | The CMM block library has run on bare metal since November 2025 and still runs today | The block store itself |
| 3 | **The fingerprint and the VIN** | Every block carries its own SHA-256 fingerprint and its own identity number, so no block can be altered or backdated | Section 3; run the script yourself |
| 4 | **The May 8, 2026 measurements** | Cache misses −98%, CPU cycles −97.5%, power −82.4%, peak memory −95.3%, taken before any patent filing | Section 3 |
| 5 | **The only proposal** | February 17, 2026: the format, the 64-byte block, the Android rollout, delivered to nine Alphabet channels | Section 4 |
| 6 | **The patent applications** | CMM, priority May 14, 2026; II-II, priority May 28, 2026; both before Google's release | USPTO 19/700,381 and 19/710,080 |

| # | Layer | What it shows | Where to check it |
|---|---|---|---|
| 7 | **Google's release** | June 5, 2026: Gemma 4 and LiteRT-LM, released to the world under Apache 2.0 | Google's own blog |
| 8 | **Google's own technical report** | Per-layer embedding tables, per-channel int2/int4 packing, fixed blocks, int8 KV, shared KV, a drafter that reads the main model's state | arXiv:2607.02770, Section 7 |
| 9 | **Google's own code** | Position addressing, fixed-width records, bit-level packing, prefetch at a computed address, memory-mapped container, integer-step state, verify-and-discard | `google-ai-edge/LiteRT-LM`, Section 8 |
| 10 | **Google's own issue boards** | Google's developers and maintainers describing a runtime that maps, reads in place by offset, rewinds by position and skips the parse pass | Section 9 |
| 11 | **Google's own counsel** | July 24, 2026: "Google respectfully declines your proposal," naming GRUS's pending applications | Section 11 |
| 12 | **Google's own procurement system** | Invoice and Utility Service Agreement acknowledged as Cases 02845835, 02856686 and 02856687 | Section 12 |
| 13 | **The public record of the bill** | Tariff PUCT Control No. 59976-1; FERC Accession 20260716-5137; account stated recorded at Parker County OPR Doc. 202618373 | Section 15 |
| 14 | **The physics** | Reductions of this size, in every physical measure at once, come from one place only: bytes that were never data, removed from the read path | Section 14 |
| 15 | **Google's own commit history** | The 64-byte assumption, the offset arithmetic, the int4 packing, the prefetch and verify-then-discard all committed after February 17, 2026; the engine rebuilt so all readable structure sits outside it, finished June 4, the day before release | `git log` on Google's repository, Section 8.4 |
| 16 | **Google's own word for it** | June 5, 2026: "a novel quantization format," signed by two named Google DeepMind staff | Section 10.4 |

Sixteen layers. Google's own report, Google's own code, Google's own commit history, Google's own boards, Google's own announcements, Google's own counsel and Google's own procurement system make up more than half of them. None of those depends on anything GRUS says.

---

## 1. The method, as GRUS defined it before June 5

### 1.1 The Compressed Memory Manifold

The Compressed Memory Manifold, USPTO 19/700,381 (provisional 64/065,881, priority May 14, 2026), is defined by what the stored state does **not** carry. A format carrying any one of the following seven classes is not CMM:

| Class | Structural element | Examples |
|---|---|---|
| (a) | Lexical delimiters | quotes, brackets, braces, parentheses, colons, commas, semicolons, equals signs, dashes, dots, slashes, structural whitespace |
| (b) | Embedded keys | field names stored in the data |
| (c) | Pointers and pointer arrays | offset tables, indirection structures, reference maps |
| (d) | Instruction maps | headers, footers, index regions, dictionary pages, string-interning tables |

| Class | Structural element | Examples |
|---|---|---|
| (e) | Schema descriptors and metadata | type prefixes, field tags, length markers, tag-length-value sequences, schema versions |
| (f) | Formatting punctuation | ASCII text for values that could be binary; whitespace for readability |
| (g) | Other self-describing elements | anything that tells a reader where data is or how to read it |

The reader knows the layout before it reads a byte. That knowledge lives in the reader's compiled code, not in the data. Records are fixed-width and aligned to the processor's 64-byte cache line. The address of record *i*, field *f* is arithmetic:

```
address = base + i × record_width + offset(f)
```

The stored bytes are mapped into memory and read where they sit. Nothing is scanned, nothing is looked up, nothing is parsed. The application states expressly that the classification is "unchanged by the selection of production method or retrieval method" (§3.4).

Every mainstream format's own specification places it outside that definition:

| Format | Classes present | Specification |
|---|---|---|
| JSON | (a), (b) | RFC 8259: six structural characters `[ { ] } : ,`; member names stored as strings |
| Protocol Buffers | (e) | Encoding guide: every field a record of "field number, a wire type and a payload" |
| FlatBuffers | (c), (e) | Internals: every table begins "with an soffset_t to a vtable" |
| XML | (a), (b), (g) | tags and attributes around every value |
| CSV | (a) | commas and line breaks |
| MessagePack, CBOR | (e) | type prefix bytes on every value |
| Avro, Parquet | (d), (e) | embedded schemas, headers, footers, dictionary pages |
| Cap'n Proto | (c) | pointer words |
| **CMM** | **none** | every value at a position known to the reader in advance |

### 1.2 Intelligent Intelligence Class II

Intelligent Intelligence Class II, USPTO 19/710,080 (provisional 64/077,191, priority May 28, 2026), is the computing substrate that runs on CMM. Its constitutive principles, in the application's terms:

- **Substrate Determinism:** "cognitive state stored in fixed-offset formats that eliminate runtime map resolution."
- **Grounded Composition:** "if an output cannot be structurally grounded, the system yields a null result."
- **Genome Authority:** a sealed, fixed core that outranks acquired data.
- **Structural determinism:** the same input yields the same output.
- **Entropy Gating:** evaluation before formulation; high-entropy branches are excluded before they are computed.
- **Anchoring:** the running system checks itself against a fixed cryptographic identity.

In operating terms: a fixed kernel pack held in place as ground truth, short-term delta lanes carried beside it, gating before computation, and branches that fail verification discarded rather than carried forward.

### 1.3 Built from real parts

Nothing in CMM is exotic hardware, and it could not be, or it would not run on a real machine. ARM cores, NEON vector lanes, 64-byte cache lines, memory-mapped files and SHA-256 all existed before GRUS. Engineering is combining parts that already exist in a way no

one has combined them. A combustion engine did not invent combustion or the cylinder. The invention is the arrangement: a record that carries none of the seven classes of self-description, sized to the cache line, read in place by position, under a fixed core that verifies before it keeps. That arrangement is what both applications claim, and it is what Google's shipped stack does.

---

## 2. GRUS built it first: the 2025 record

This was never an idea on paper. GRUS and its technology licensee, Data Incorporated, built working systems through 2024 and 2025, ran them, and published as they went.

| Date | Record | Verify |
|---|---|---|
| 2024 | RecursiveMind, the recursive-memory AI app, hosted on Google's own Firebase; still live at its Google-issued address | studio--recursivemind.us-central1.hosted.app |
| Mar 23, 2025 | A cryptographic hash bound to a running AI as its identity anchor | ledger, recomputable today |
| Mar 27, 2025 | Hash-chained ownership ledger entry; transaction hash `3a6cc989…fad3`, recomputable from its committed values | SHA-256 of the committed values |
| Apr 5, 2025 | Runtime execution identity locked | ledger |
| **May 8, 2025** | **GRUS Total Intellectual Lockdown License issued** | chloe-runtime LICENSE |
| **May 12, 2025** | **Class I.I. repository created, unchanged since; strict no-reuse, no-fork, no-derivative license describing CMM** | Class I.I. |
| May 2025 | chloe-runtime grows from 1,256 to 260,877 characters over 18 commits | commit history |
| Jun 21, 2025 | Phoenix Mark I: a Verilog co-processor with an entropy-threshold unit and a curl-gradient processor, timed for 3.5 GHz, carried the same day into deterministic code verified against golden test vectors | GRUS record |
| 2025 | A C kernel with an Android NDK bridge running the core math as native code on the phone's ARM processor, with an ARM NEON path for the inner loops | GRUS record |
| Jul 2025 | First standing case with the National Intellectual Property Rights Coordination Center | NIPRCC |
| Jul 12, 2025 | Engine 18 node created on Google Cloud | Google Cloud instance record |
| Aug 2025 | Anchored Intelligence deployed; public Jailbreak Challenge on Google's own hosting | post |
| **Sep 5, 2025** | **Digital Physics Charter published: ten laws for digital machines** | Charter |
| **Sep 15, 2025** | **Class II, Entropy-Lock and the CMM memory system tied together publicly, with a 50× storage-density advantage** | post |
| Sep 19, 2025 | Request filed with Google and acknowledged (request 5-6125000038710) | Google's own acknowledgment |
| Oct 14, 2025 | Second standing NIPRCC case | NIPRCC |
| Oct 19, 2025 | A 64-byte CMM genome block minted for the GRUS Google Cloud environment snapshot, sealed by CRC and leaf hash | block record |

| Date | Record | Verify |
|---|---|---|
| **Nov 2025 → today** | **The CMM block library running continuously on bare metal** | Section 3 |

GitHub's server-side `created_at` and `pushed_at` dates are written by GitHub, not by GRUS. They cannot be edited by the repository owner. A repository created on May 12, 2025 and unchanged since is a dated witness that the expression it holds existed on that date.

Copyright attaches the moment original work is fixed (17 U.S.C. § 102(a)). The 2025 code, specifications, lane definitions and license are GRUS's expression, fixed a year before Google's release. The patent applications reach the method. The copyrights reach the expression. They are two separate locks on the same work, and Google's 2026 license conveys nothing in either.

---

## 3. The running system, its fingerprint and its VIN

### 3.1 Running since November 2025

The CMM block library has run continuously on GRUS's own desktop, bare metal, since November 2025. It boots with the computer, backs up its blocks every day, and can be opened live today in VS Code. The library is the same or larger now than it was when it was measured. The November 21, 2025 deterministic reasoner reads 64-byte blocks, each validated by checksum and SHA-256 on load, with no model inference at all.

The machine is an Intel i9-13900K with 64 GB of DDR5. Its L1, L2 and L3 caches all move 64-byte lines. GRUS's records are sized to that line, so one fetch delivers one whole record.

### 3.2 The fingerprint and the VIN

A car's VIN is stamped on the frame. It identifies the vehicle, and anyone can check it against the title. Every CMM block carries the same kind of identity, and it is stamped into the bytes.

A directory listing of the store returns:

```
Found 137337 .block files in: C:\Users\1\.chloe_core\cmm_blocks
```

Each block is 96 bytes, laid out by position with nothing to describe it:

| Bytes | Lane | Content |
|---|---|---|
| 0–3 | **VIN** | 4-byte block index |
| 4–31 | **Fingerprint** | 28-byte anchor = the first 28 bytes of SHA-256 of the payload |
| 32–95 | Payload | 64 bytes, one cache line |

The file name carries the same identity a second time: the block index in 8 hex digits, followed by the anchor. Two blocks from the populated-block exhibit of record, exactly as stored:

```
library-OpenCV-0006-000000062177eb391a1bcc87cb5d5a05.block
AAAABiF36zkaG8yHy11aBWGfx9F4/dbrlyCiO9A9hxVjYWwgZmxvdywgb2JqZWNOIHRyYWNraW5nCjYuIEltYWdlIHNOaXRjaGluZyBmb3IgcGFub
3JhbWFzCjcuIEZh

library-OpenCV-0007-00000007f7b361e7c09f74aebcebc8c7.block
AAAAB/ezYefAn3SuvOvIx461ejOuXfwzTMueM/m25exjZSByZWNvZ25pdGlvbiBhbmQgZGVOZWNOaW9uCjguIE1hY2hpbmUgbGVhcm5pbmcgbW9kd
WxlIChTVkOsIGRl
```

Run this on either one:

```
import base64, hashlib
block =
base64.b64decode("AAAABiF36zkaG8yHy11aBWGfx9F4/dbrlyCi09A9hxVjYWwgZmxvdywgb2JqZWN0IHRyYWNraW5nCjYuIEltYWdlIHN0aXR
jaGluZyBmb3IgcGFub3JhbWFzCjcuIEZh")
vin, anchor, payload = block[0:4], block[4:32], block[32:96]
print("VIN        ", vin.hex())
print("stored     ", anchor.hex())
print("SHA-256    ", hashlib.sha256(payload).hexdigest()[:56])
print("match      ", anchor == hashlib.sha256(payload).digest()[:28])
```

Output:

```
VIN         00000006
stored      2177eb391a1bcc87cb5d5a05619fc7d178fdd6eb9720a2d3d03d8715
SHA-256     2177eb391a1bcc87cb5d5a05619fc7d178fdd6eb9720a2d3d03d8715
match       True
```

Block 7 returns `00000007` and `f7b361e7c09f74aebcebc8c78eb57a33ae5dfc334ccb9e33f9b6e5ec`, matched the same way. The file name tail `000000062177eb391a1bcc87cb5d5a05` is the VIN followed by the start of the fingerprint.

That is what makes the store unforgeable. Change one byte of a payload and its fingerprint no longer matches. Fabricate a block after the fact and you have to forge SHA-256. Each block is a self-verifying leaf: the fingerprint proves the content, the VIN fixes its position, and the file system dates it. The filed II-II application recites the 64-byte CMM block with fixed-offset lanes, including an ANCHOR lane and an identity-leaf lane. The 2025 public repository recites the lanes DELTA16, ANCHOR8, POLICY64, SKETCH16 and VQ16. The anchor shown above is that lane, populated with the payload hash, exactly as the filed and published structure specifies.

The block itself is the method. Its payload is one cache line. Its identity is read by position, at byte 0. Its fingerprint is read by position, at byte 4. Its content is read by position, at byte 32. There is not one delimiter, key, pointer or schema byte in it.

### 3.3 The May 8, 2026 measurements

The performance figures in both patent applications were pulled from that running system on May 8, 2026, before either application was filed, at full capacity after the library had grown. They were taken on 500,000 financial transaction records against the same data in JSON:

| Measure | JSON | CMM | Change |
|---|---|---|---|
| Storage | 35.0 MB | 12.5 MB | −64.3% |
| Cold load | 895 ms | 139 ms | 6.44× faster |
| Peak RAM | 850 MB | 40 MB | −95.3% |
| L1 cache miss rate | 40.2% | 0.81% | −98% |
| CPU cycles | 1.2 billion | 30 million | −97.5% |
| Thermal load | 125 W | 22 W | −82.4% |

These are hardware counters: cache-line fills, retired instructions and watts. They are physical events, not feature descriptions. Every one of them measures the same thing from a different side: bytes that were never data stopped being stored, moved, fetched and scanned.

---

## 4. The only proposal

On February 17, 2026, GRUS sent Alphabet a *Confidential Strategic Proposal* titled *"5-Year Infrastructure Transformation & Resource Optimization Strategy."* It was addressed to Alphabet's Office of the CFO and SVP of Technical Infrastructure and delivered through nine

Alphabet channels, including the Offices of the CEO, the CFO, the CEO of Google Cloud, the General Counsel and the IP Licensing group. It carried a 72-hour exclusivity window that closed February 20 without a response.

Its own terms said what it was, on page 7: *"This proposal is not a product sale. This is a utility partnership."*

What it put in Google's hands, in its own words:

- **The format:** CMM, *"a universal data format that replaces human-readable storage architectures (JSON, XML, uncompressed logs) with a compact, binary-encoded structure,"* built on a *"Fixed 64-byte block structure"* with *"Binary vector encoding (eliminates JSON/XML text overhead)"* and *"Vector quantization indices."*
- **Phase 1:** convert Google's storage to CMM and cut active rack count.
- **Phase 2:** replace internal transport so data stops costing CPU cycles at every node.
- **Phase 3, "Distributed Inference Architecture":** *"Shift AI inference workload from centralized servers to distributed client devices,"* activating models *"on Android devices"* through a *"Play Services update,"* with *"Client NPUs (Neural Processing Units) handl[ing] 90% of inference tasks"* across Google's *"3B+ Android"* devices.
- **The delivery:** a complete CI/CD conversion pipeline ready to run, converting Google's cloud files to CMM in about 90 days at no engineering cost to Google.
- **The reason:** Google's $185 billion 2026 capital plan and the energy it would take to power it.

That is the method, the hardware target, the rollout order and the economics, delivered to Google's general counsel's office four months before June 5.

It is the only proposal GRUS has ever made to Google. Everything GRUS sent after June 5 was an invoice, a service agreement for service already in use, or a notice. GRUS has never asked Google for a royalty, a deal, a license negotiation or a buyout, before June 5 or after. The position has not moved from the first day: *you are using it; here is the service agreement; here is the bill.*

---

## 5. The filings and the public announcement

| Date (2026) | Record |
|---|---|
| May 2026 | CMM described publicly on Wikipedia until the article was removed (revision record) |
| **May 14** | **CMM provisional 64/065,881 filed**, and announced the same day: *"Establishing the Standard: Green Recursive Utility Service LLC Secures the Future of Grid-Relief Data Architecture"* (Medium). The post described CMM as eliminating "the massive bloat of traditional data routing (JSON, pointers, lexical maps)" through "delimiter-free state rehydration," gave the measured 82% thermal-load reduction and 139 ms cold start, and carried a patent-pending notice stating that no license was granted |
| May 26 | Copper-gold alloy provisional 64/074,405 filed |
| **May 28** | **II-II provisional 64/077,191 filed** |
| Jun 8 | CMM non-provisional 19/700,381 filed, priority May 14 |
| Jun 16 | II-II non-provisional 19/710,080 filed, priority May 28 |
| Early Oct | USPTO notice: every II-II provisional deficiency met |

Assignee: Green Recursive Utility Service LLC, Texas SOS 806584578. Both applications are of record at patentcenter.uspto.gov. The provisional windows run into May 2027, and every correction, supplement or refiling ties back to the May 14 and May 28, 2026 priority dates.

As between two applicants, priority runs by filing order (35 U.S.C. § 102). Google executed an inventor's declaration before the USPTO on June 22, 2026. GRUS's priority dates are May 14 and May 28. GRUS is first.

---

## 6. June 5, 2026: the release

On June 5, 2026, Google DeepMind released the Gemma 4 quantization-aware models and the LiteRT-LM runtime to the world under Apache 2.0: "we've reduced the memory footprint of Gemma 4 E2B to 1GB" (Google). NVIDIA was Google's partner in the deployment; the two had announced joint optimization of Gemma 4 across NVIDIA's RTX GPUs, data-center hardware and Jetson edge devices on April 2 (NVIDIA).

GRUS posted notice on Google's own tracker within 24 hours.

Everything Google shipped after that date is set out below, in Google's own words.

### 6.1 NVIDIA, the joint partner

This was not one company's release. It was a joint rollout, and NVIDIA said so in its own words:

- **April 2, 2026:** "Google and NVIDIA have collaborated to optimize Gemma 4 for NVIDIA GPUs," across RTX GPUs, data-center hardware and Jetson edge devices (NVIDIA).
- **June 8, 2026:** Apple's research team: "we worked with Google and NVIDIA to extend Private Cloud Compute to NVIDIA GPUs in Google Cloud" (Apple). All three partners in one sentence.
- **June 12, 2026:** NVIDIA's DiffusionGemma, built on Gemma 4, "up to 1,000 tokens/sec" on H100 (NVIDIA).

NVIDIA received GRUS's notice by certified mail and is invoiced under the same tariff. Through NVIDIA's reach across GPUs, data centers and edge devices, the joint rollout carried the method into the industry at once, and GRUS now bills 38 major companies downstream of it. A coordinated, two-company release across every class of hardware is not a coincidence and it is not something that was already lying around. It is a rollout, in the order the February 17 proposal laid out.

---

## 7. Google's own technical report

On July 2, 2026, the Gemma Team at Google DeepMind published the *Gemma 4 Technical Report* (arXiv:2607.02770). It presents the design as Google's own: "We introduce several architectural and methodological innovations." It came out four and a half months after the February 17 proposal and seven weeks after the CMM filing.

The report states, in Google's words, what the models store, at what width, and how the pieces read one another. The shipped runtime in Section 8 shows how those stored bytes are read. Each statement below is quoted from the report and set beside the line of Google's code that reads it.

### 7.1 Tables recalled by identity: per-layer embeddings

> "E2B and E4B use per-layer embeddings as in Gemma 3n (Gemma Team, 2025b), making them 2.3B and 4.5B effective out of 5B and 8B total parameters respectively."

> "Note that the extra embedder parameters in E2B and E4B are per-layer embeddings." (Table 1: E2B embedder **400M + 2,340M**; E4B embedder **670M + 2,820M**)

More than half of E2B's parameters are not computed on each token. They sit in tables, one per layer, and the row a token needs is fetched by the token's identity. That is recall from a fixed table by numerical offset. Google's runtime does it in exactly those terms:

```
size_t row_offset = id * row_size_bytes;
const uint8_t* row_data = table + row_offset;
absl::PrefetchToLocalCache(table_ptrs[table_idx + 1] + row_offset);
```

( `runtime/executor/llm_litert_npu_compiled_model_executor_utils.cc` )

The row's location is the token's identity times a fixed row width. There is no delimiter between rows, no header on a row, no offset table and no schema map. `table_ptrs` is one table per layer, and the next layer's row is pulled into cache at the same computed offset before it is asked for, because the layout is known before the read. Google's report names the tables. Google's code reads them by `base + i × width`.

### 7.2 Values packed into fixed bit positions: int2 and int4

> "mobile quantization: per-channel low bitwidth weight (mix of int2 and int4) and activation quantization (int8)."

> "Q4_0 quantization: blockwise quantization, often referred to as Q4_0."

> On the audio encoder, "we further reduce activation precision to 8 bits and weight precision to {2, 4, 8} bits, varying by layer cluster."

An int4 value is half a byte. An int2 value is a quarter of one. Nothing inside the byte says which value is which; the reader knows, because the width is fixed in advance. Google's runtime recovers them by position and shift:

```
(high << 4) | low      // PackInt4 / UnpackInt4Row: two 4-bit values per byte
```

( `runtime/executor/llm_litert_npu_compiled_model_executor_utils.cc` )

Q4_0, the block format the report names, is the llama.cpp layout in which every block is the same size: one scale and thirty-two 4-bit values, 18 bytes, with no tag, no length and no key (ggml block_q4_0). Block *n* sits at `n × 18`. That is bit recall off a numerical offset through a fixed width, in the formats Google chose and named.

### 7.3 Fixed-width chunks, position carried by order

> "Raw audio is segmented into 40ms chunks at 16kHz, resulting in 640-dimensional vectors per chunk. These are projected directly into the LLM embedding space. Since audio is a temporal sequence, it does not require additional positional encoding."

Every chunk is the same width. Its position is its place in the sequence, not a field written beside it. That is the CMM principle in Google's own sentence: where the reader already knows the layout, nothing has to describe it.

> The encoder-free 12B model takes "48 × 48 × 3 RGB patches," replacing the 550M vision encoder "by a single large matmul," which alleviates "the need for separate encoders and reduc[es] memory fragmentation."

Fixed-size patches, no encoder in between, less fragmentation. Memory laid out in fixed grains is memory that does not fragment.

### 7.4 The KV cache: fixed width, shared, and smaller

> "Extended contexts lead to a memory explosion in the KV cache."

> "We improve memory efficiency by re-using keys as values in the global attention layers … i.e., values = keys. … effectively reducing the global KV cache by 37.5%. … Finally, we share the KV cache with ratios of 20/35 and 18/42 for the E2B and E4B model."

> Table 3: memory "for weights and int8 KV caching (+KV) at 32k context size": E2B **4.6 GB** in bf16, **0.8 GB** quantized, **+0.05 GB** KV.

An int8 KV cache is a fixed-width array. Sharing it across 20 of 35 layers means one stored state read by many readers at known positions, instead of copies. Google's runtime addresses that state the same way:

```
lock.second[start_pos * hidden_dim + i];          // position × fixed width + lane
lock.second[h * cache_seq + start_pos];           // transposed: head × length + position
```

( `runtime/executor/llm_litert_npu_compiled_model_executor_utils_test.cc` )

That is II-II's Substrate Determinism in Google's code: "cognitive state stored in fixed-offset formats that eliminate runtime map resolution."

### 7.5 The drafter reads the main model's state in place, then is verified

> "In our MTP procedure, the model's last layer activations from the previous step and token embeddings are fed into the MTP head. The MTP head generates future tokens sequentially using a separate embedder and a 4-layer Transformer block that cross-attends to the KVs of the main model (Figure 1), thus eliminating the need for MTP prefill and supporting any draft length."

The drafter does not rebuild the main model's memory. It reads the main model's stored state where it sits. Google's runtime locates the verified record by index times fixed width, then keeps only what the base model verifies:

```
size_t offset = last_verified_token_id_idx * chunk_size_bytes;   // llm_litert_mtp_drafter.cc
// kVerifySignatureRunner = "verify"; on divergence, break, then:
drafted_tokens.resize(num_correct_tokens);
```

The fixed core holds sole authority; the drafter holds none and is constructed with a `CompiledModel& base_model_` it does not own. Unverified output is cut off before it becomes output. That is II-II's Genome Authority and Grounded Composition: a sealed core that outranks what is acquired, and a null result for anything that cannot be grounded.

### 7.6 Gating before computation

> "For the E2B and E4B drafters, we reduce the decoding overhead by replacing the projection operation to the entire vocabulary by a top-k operation on clusters of tokens. As a result, final matrix multiplication is reduced from d × 262,000 to d × 4096 while preserving a similar acceptance rate."

The projection over all 262,000 tokens is never run. A top-k over clusters decides first, and only what passes that gate is computed. Candidates are excluded before the multiplication runs, not filtered after. That is Entropy Gating: evaluation prior to formulation. Google's runtime does the same thing in the decoder:

```
// ConstrainedDecoding::Decoder and ::Constraint linked into the core pipeline
// HWMaskUpdate writes masked_val into invalid positions before computation
```

### 7.7 The physical results Google reports

> "On the 150M image encoder, quantizing activations and weights to 8-bit precision (W8A8) yields a 2× reduction in total forward-pass memory footprint (from 400 MB to 200 MB, including on-device compilation overhead) and a 44% reduction in on-device latency relative to Gemma 3n on newer hardware."

> "Overall, we achieve a 78% reduction in on-disk footprint, from 390 MB in Gemma 3n to 87 MB in this version."

> "We improve inference speed, memory, and compute efficiency, as well as long-context abilities through critical design choices."

Memory, latency, disk and compute, all down at once, presented as "innovations" in a report dated July 2, 2026.

### 7.8 What the report adds up to

| Google's report says | Google's runtime reads it | CMM / II-II element |
|---|---|---|
| Per-layer embedding tables, 2,340M parameters in E2B | `row_offset = id * row_size_bytes`; `table + row_offset`; prefetch next layer at same offset | Recall by identity at a computed offset; no keys, no pointers, no map |
| Per-channel int2 / int4 weights; Q4_0 fixed blocks | `(high << 4) \| low`; block *n* at `n × 18` | Fixed-width values recovered by position and shift; nothing self-describing |
| 40 ms chunks, 640-dim vectors, no positional encoding needed | Sequence order is position | Layout known in advance; position not written in the data |
| int8 KV cache, shared 20/35 and 18/42, keys reused as values | `start_pos * hidden_dim + i` | Cognitive state at fixed offsets; one state, many readers |
| Drafter cross-attends to the main model's KVs, no prefill | `last_verified_token_id_idx * chunk_size_bytes`; `resize(num_correct_tokens)` | State read in place; verify, then discard |
| Projection cut from d × 262,000 to d × 4096 | `HWMaskUpdate` masks before compute | Gating before computation |
| 2× memory, 44% latency, 78% disk | — | The physical result of bytes not moved |

Every row is Google describing, in its own technical paper, a stored state of fixed widths read by position. Every row has a matching line in Google's own source. Every row is an element GRUS defined, ran, measured, offered to Google and filed on first.

---

## 8. Google's own code

Every excerpt below is from Google's public `google-ai-edge/LiteRT-LM` repository, release v0.17.1, or from threads on Google's own trackers. Anyone can open the same files and read the same lines.

### 8.1 Element by element

| CMM / II-II element | Google's own code or record | What it does |
|---|---|---|
| **Fixed 64-byte grain** | `// We assume capacity is a multiple of 64 for all models.` (`runtime/executor/llm_litert_npu_compiled_model_executor_utils.cc`) | The structural unit is sized to the cache line by assumption, not described in the data |
| **Fixed-width lanes for the processor** | `hidden_dim = 32; // multiple of 16 for NEON`, with test sizes 1027, 1033 and 1041 chosen as non-multiples of 4, 8 and 16 only to exercise the scalar tail (`…_utils_test.cc`) | The primary path processes hardware-aligned, fixed-width blocks |
| **Address by position, not by key** | `size_t row_offset = id * row_size_bytes;` `const uint8_t* row_data = table + row_offset;` (`…_utils.cc`) | No delimiters between records, no per-record headers, no offset table, no schema map. A row's location is computed from its identity alone |
| **Fixed-width records, fields by offset** | `size_t offset = last_verified_token_id_idx * chunk_size_bytes;` (`runtime/executor/llm_litert_mtp_drafter.cc`) | Index × fixed record width; two fields at offsets 0 and `chunk_size_bytes`, nothing between them |
| **State addressed by position** | `lock.second[start_pos * hidden_dim + i]`; transposed `lock.second[h * cache_seq + start_pos]` (`…_utils_test.cc`) | KV state read at computed offsets |
| **Bit-level recall by position** | `PackInt4` / `UnpackInt4Row`: `(high << 4) \| low` (`…_utils.cc`) | Values packed into fixed bit positions and recovered by shifting |
| **Prefetch at a computed address** | `absl::PrefetchToLocalCache(table_ptrs[table_idx + 1] + row_offset);` (`…_utils.cc`) | The next record pulled into cache from an address calculated in advance |
| **Container opened by mapping** | `LMLoader` links `MemoryMappedFile` and `Schema::Core`; `FileFormatUtil` links `MemoryMappedFile` (`runtime/util/CMakeLists.txt`) | The model file is mapped into memory, not deserialized |
| **Sections loaded by known boundaries** | `enable_file_backed_model_loading` on `Backend::NPU`; `GetParallelFileSectionLoading()` (`runtime/core/engine_advanced.cc`) | Parallel section loading requires every boundary to be known before anything is read |
| **Fixed-layout binaries** | `prebuilt/`: eight platform targets, exclusively 64-bit | Shipped as fixed-layout machine code |

| CMM / II-II element | Google's own code or record | What it does |
|---|---|---|
| **State is a position counter** | `checkpoint_map_[label] = {current_step, session_state_, last_task_ids_};` ( `runtime/core/session_advanced.cc` , line 487) | A checkpoint is one integer step, a state flag and task IDs. Nothing is copied and nothing is described |
| **Rewind is arithmetic on that counter** | `return execution_manager_lock->SetCurrentStep(*session_info_, target_step);` ( `session_advanced.cc` , lines 517 and 542) | Moving backward through the model's memory means setting one number. No parse, no reload |
| **No self-description in the core** | `std::optional<int> checkpoint_message_index_` : "The index of the message you have to rewind to in order to remove channel content from the KV cache" ( `runtime/conversation/conversation.h` , lines 792–794) | Engine state addressed by an integer index; the history lives outside it, in `std::vector<Message> history_` (line 774) |
| **Readable structure only at the edge** | `#include "nlohmann/json_fwd.hpp"` ( `conversation.h` , line 18); Jinja `"jinja_prompt_template"` (lines 288–289); prefill text "is obtained by taking the difference between the rendered string…" (lines 719–722) | JSON, templates and text diffing are confined to the conversation layer. What crosses into the engine is an `InputData` vector for `Session::RunPrefill` (lines 736–751) |
| **Structure rebuilt outside the core** | `RewindAndGetInputDataVector` : rewinds the session, then returns input "for all messages from that point onward" ( `conversation.h` , lines 753–756) | When structure is needed again, the outer layer re-renders and re-feeds it, because the core never carried it |
| **Verify, then discard (II-II)** | `kVerifySignatureRunner` = "verify"; on divergence, `break` , then `drafted_tokens.resize(num_correct_tokens);` | Unverified output cut off before it becomes output |
| **Fixed core holds sole authority (II-II)** | Drafter constructed with `CompiledModel& base_model_` it does not own; the base model alone verifies | Genome Authority |
| **Deterministic recall (II-II)** | `set_k(1); set_p(0.0f); set_seed(0);` (greedy sampler); `FindMaxIndex` returns the first occurrence on ties across scalar, NEON and SSE2 | The same input yields the same output on every processor |
| **Gating before computation (II-II)** | `ConstrainedDecoding::Decoder` and `::Constraint` linked into the core pipeline; `HWMaskUpdate` writes `masked_val` into invalid positions before computation | Disallowed branches masked out before they are computed |

Read top to bottom, the table describes one system: a core that carries no description of itself, sized to the cache line, addressed by position, packed to fixed bit widths, read in place from mapped memory, rewound by changing a number, governed by a fixed core that verifies before it keeps, with readable structure pushed out to the edge and rebuilt there when anyone needs it. That is the Compressed Memory Manifold with Intelligent Intelligence Class II running on it.

### 8.2 Verify it yourself

Every line above can be checked against Google's public repository in under two minutes:

```
GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/google-ai-edge/LiteRT-LM.git
cd LiteRT-LM
```

```
# Fixed-offset record access
grep -rn "row_offset" runtime/executor/
grep -rn "last_verified_token_id_idx \* chunk_size_bytes" runtime/executor/
grep -rn "start_pos \* hidden_dim" runtime/executor/

# Hardware alignment
grep -rn "multiple of 64" runtime/executor/
grep -rn "multiple of 16 for NEON" runtime/executor/

# Sub-byte packing
grep -rn "UnpackInt4Row\|high << 4" runtime/executor/

# Prefetch at a computed offset
grep -rn "PrefetchToLocalCache" runtime/executor/

# Memory-mapped container, sections by known boundary
grep -rn "MemoryMappedFile" runtime/util/CMakeLists.txt
grep -rn "enable_file_backed_model_loading\|GetParallelFileSectionLoading" runtime/core/

# State as an integer step; rewind by setting it
grep -n "checkpoint_map_\|SetCurrentStep" runtime/core/session_advanced.cc

# Readable structure confined to the conversation layer
grep -n "nlohmann\|jinja\|history_\|checkpoint_message_index_\|RewindAndGetInputDataVector"
runtime/conversation/conversation.h

# Verify, then discard
grep -rn "kVerifySignatureRunner\|resize(num_correct_tokens)" runtime/executor/

# Deterministic sampling
grep -rn "CreateGreedySampler" -A8 runtime/executor/

# Gating before computation
grep -rn "ConstrainedDecoding" runtime/core/CMakeLists.txt
grep -rn "masked_val" runtime/executor/
```

GRUS's full source-level analysis of LiteRT-LM is deposited on Zenodo (record 21482573), and the fifteen-artifact source-to-specification mapping is published in full in *One Proposal* (10.5281/zenodo.23300835). The mapping stopped at fifteen not because the correspondence ran out, but because the point was made. The repository was not exhausted.

### 8.3 Removed, not compressed: the overhead sits outside Google's engine

This is the June breakthrough, and it is in Google's own repository.

Delimiters, keys, pointers and schema are physical bytes. They cannot be compressed out of existence. Lossless compression gives every one of them back the moment the data is used (Section 14.3). Fewer bits per tensor value shrink the values; they do not touch the bytes around the values. However good the embedding tables or the tensor formats are, the only way to stop paying for the overhead is to take it out of the path the processor reads.

That is exactly what Google's runtime does. Google's own code splits into two layers:

| **Outside the engine: the conversation layer** | **Inside the engine: the session** |
|---|---|
| `#include "nlohmann/json_fwd.hpp"` : JSON ( `conversation.h` , line 18) | `checkpoint_map_[label] = {current_step, session_state_, last_task_ids_};` : state is an integer step ( `session_advanced.cc` , line 487) |
| `"jinja_prompt_template"` : text templates (lines 288–289) | `SetCurrentStep(*session_info_, target_step)` : rewind is setting one number (lines 517, 542) |

| Outside the engine: the conversation layer | Inside the engine: the session |
|---|---|
| Prefill text "obtained by taking the difference between the rendered string…": string diffing (lines 719–722) | Receives an `InputData` vector through `Session::RunPrefill` (lines 736–751) |
| `std::vector<Message> history_` : the readable history (line 774) | Carries no history, no keys, no schema: `checkpoint_message_index_` is an integer (lines 792–794) |
| `RewindAndGetInputDataVector` : re-renders and re-feeds structure when it is needed again (lines 753–756) | Read in place from memory-mapped storage, by position |

Every one of the seven classes of overhead sits on the left. None of it is on the right. Google did not compress the delimiters. It removed them from the engine and set them outside, at the edge, where a person reads them. The engine runs clean, positional and self-describing in nothing. That is the Compressed Memory Manifold: the stored state carries none of the seven classes, and readable structure is confined to the edge.

An outside developer saw the same split from the other side and wrote it down on Google's own board: rewinding the engine "does not restore the conversation's message history, append state, or channel-filtering bookkeeping," because the engine never carried them (Section 9).

### 8.4 When Google wrote it: Google's own commit history

Google's repository records when each line went in. Cloning it and running `git log -G"<line>"` returns the first commit that introduced it. These lines were committed after GRUS's February 17, 2026 proposal reached nine Alphabet channels:

| Line in Google's code | First committed | Commit |
|---|---|---|
| `checkpoint_map_[label] = {current_step, …}` : engine state as an integer step | **Mar 23, 2026** | `018c45da` |
| `size_t offset = last_verified_token_id_idx * chunk_size_bytes;` , `kVerifySignatureRunner` , `num_correct_tokens` : fixed-width records; verify, then discard | **Mar 24, 2026** | `0fcef0c1` |
| `GetParallelFileSectionLoading()` : sections loaded by known boundaries | **Apr 2, 2026** | `1e0828e8` |
| `RewindAndGetInputDataVector` , `checkpoint_message_index_` , `GetPrefillTextForMessages` : readable structure rebuilt outside the engine | **Apr 2, 2026** | `81a7962c` |
| `// We assume capacity is a multiple of 64 for all models.` ; `masked_val` : the 64-byte grain; gating before computation | **Apr 23, 2026** | `17aca0be` |
| `hidden_dim = 32; // multiple of 16 for NEON` : fixed-width lanes | **Apr 23, 2026** | `4d288b56` |
| `PrefetchToLocalCache` , `UnpackInt4Row` : prefetch at a computed address; bit-level recall by position | **Apr 28, 2026** | `cc1d024b` |
| "Extract SessionInterface from Engine": the engine separated out | **May 20, 2026** | `3e303927` |
| `enable_file_backed_model_loading` : file-backed loading on the NPU | **May 28, 2026** | `2c9c1de3` |
| "Add `RewindToStep` function to `SessionAdvanced` ": rewind by position | **Jun 4, 2026, the day before release** | `0147ffd4` |
| "Strip blobs from PromptTemplateInput before applying template": material stripped before it reaches the engine | Jun 8, 2026 | `5353c8f6` |
| `size_t row_offset = id * row_size_bytes;` : a row found by identity times fixed width | Jun 11, 2026 | `94239b94` |

Two things are in that table, and they are different things.

**The 64-byte structure is the marker.** "We assume capacity is a multiple of 64 for all models" went into Google's code on April 23, 2026. GRUS's February 17 proposal had put a "Fixed 64-byte block structure" in front of Google's general counsel's office nine weeks earlier.

The marker shows where the blueprint came from.

**The engine rebuild is the breakthrough.** From March 23 to June 4, Google turned its engine into a positional core: state as an integer, rewind by setting it, structure rebuilt outside it, the engine extracted as its own interface, `RewindToStep` added the day before the June 5 release. Then on June 5, Google called what it shipped "a novel quantization format" (Section 10.4). That is the step around the seven classes of overhead, built between the proposal and the release, in Google's own commits.

Verify it yourself:

```
git clone https://github.com/google-ai-edge/LiteRT-LM.git && cd LiteRT-LM
git log -G"multiple of 64 for all models" --reverse --format='%cs %h' -- runtime | head -1
git log -G"last_verified_token_id_idx \* chunk_size_bytes" --reverse --format='%cs %h' -- runtime | head -1
git log -G"UnpackInt4Row" --reverse --format='%cs %h' -- runtime | head -1
git log -G"RewindAndGetInputDataVector" --reverse --format='%cs %h' -- runtime | head -1
git log --format='%cs %h %s' --grep="RewindToStep" -- runtime/core
```

---

## 9. Google's own issue boards

These threads are written by Google's developers, users and maintainers, on Google's own trackers. They record how the shipped runtime actually reads its models.

- **Map, don't copy.** On Apple platforms LiteRT-LM keeps a model's weight buffer as memory-mapped file pages, read in place; on Android it copies the weights into a separate buffer. Same model, same runtime: mapped weights hold **1,603 MB** resident; copied weights hold **2,644 MB**. Google's maintainers responded and committed to investigate (LiteRT-LM #3507). Reading in place instead of moving bytes is worth 780 MB on Google's own board.
- **Read-only, read in place.** A proposed fix maps the weight file read-only because "weight data is never mutated by the runtime," tested by loading `gemma-4-E2B-it.litertlm` on an iPhone (LiteRT-LM PR #2076).
- **No parse pass on the container.** The `.litertlm` header is read straight from the buffer with `flatbuffers::GetRoot` "without calling the FlatBuffers verifier," and the generated verification function "exists but is never called anywhere" (LiteRT-LM PR #1888).
- **The default path maps and goes.** In TensorFlow Lite, the standard load path "mmaps the file and constructs the model **without** running `Verify()` " (TensorFlow #128148).
- **Cut out by byte position.** A reproduction on Google's LiteRT board takes Google's official Gemma 4 E2B bundle ( `gemma-4-E2B-it.litertlm` , checked by SHA-256) and pulls the decoder out by position alone: seek to byte **1,725,530,112**, read **818,275,184** bytes. No parser, no lookup, just an offset and a length. The runtime log then shows the GPU environment created with `enable_host_mapped_pointer=true` , followed by "threads to upload weights" and "threads to compile kernels," working directly from those bytes (LiteRT #10446).
- **When the mapping fails, the model fails.** On a Samsung Galaxy A55, Gemma 4 decodes into garbage on the GPU while the driver repeats "Valid mapped region entry not found" (LiteRT-LM #2611). The runtime depends on reading memory exactly where it expects it to be.
- **The speed is in how the bytes move, not in the math.** A developer hooked the graphics interface beneath Google's shipped LiteRT-LM web engine running Gemma 4 E2B and recorded every kernel it runs, "from the shipped package, not from source." Per token, nine kernels stream **740.6 MiB** of weights, and four quantized matrix-vector kernels (2-bit and 4-bit) take **48.5%** of GPU time, moving data at 78 to 81 GB/s, "a fifth of the chip's rated 400 GB/s." The developer changed only how each kernel splits its memory reads across threads, "arithmetic untouched," output matching. The model went from **56.7 to 70.8 tokens per second** on an Apple M2 Max, and the same change ran **1.58×** faster on an NVIDIA T4 (LiteRT-LM #3775; harness and records). Same weights, same math, same answers: the entire gain came from how the bytes were read.
- **The state rewinds by position and carries no history of its own.** A developer building on LiteRT-LM v0.15.0 and v0.17.1 asked Google for conversation-level rollback. The session primitives `SaveCheckpoint()` , `RewindToCheckpoint()` and `RewindToStep()` move "the native session position." Rewinding "does not restore the conversation's message history, append state, or channel-filtering bookkeeping," so the developer's prototype "saves session checkpoints together with copied message history and append state… It does not copy the KV tensors." On the NPU path the state is fixed to the silicon: "the specialized NPU executor only permits a one-token rollback." What the developer wants is to reuse the retained context "without reloading the engine or replaying the saved prefix."

That last thread is the II-II split, rebuilt by hand by an outside developer: a fixed core that stays where it is, and a separate record of changes kept beside it. The developer had to build the second half because Google's runtime ships only the first.

---

## 10. Google's own announcements

GRUS is not putting numbers in anyone's mouth. Every figure below was published by the company named, under its own name. Every date falls after the February 17 proposal.

### 10.1 Before June 5: where Google said it stood

| Date (2026) | Company | What it said | Source |
|---|---|---|---|
| Apr 2 | Google AI Edge | Gemma 4 E2B in "<1.5GB memory on some devices" | Google |
| Apr 2 | NVIDIA | "Google and NVIDIA have collaborated to optimize Gemma 4 for NVIDIA GPUs" | NVIDIA |
| Apr 22 | Google Cloud | TPU 8t and 8i "deliver up to two times better performance-per-watt" | Google |
| May 19 | Google LiteRT-LM | A 2.58 GB Gemma 4 E2B with "a physical memory footprint of just 607MB on Apple mobile CPUs," by loading encoders only when needed | Google |

### 10.2 June 5 onward: the release, and everyone after it

| Date (2026) | Company | What it said | February 17 proposal | Source |
|---|---|---|---|---|
| **Jun 5** | **Google DeepMind** | **"we've reduced the memory footprint of Gemma 4 E2B to 1GB,"** under Apache 2.0 with LiteRT-LM | CMM format; "Vector quantization indices"; on-device inference | Google |
| Jun 5 | Unsloth | The new variants "reduce memory requirements around 3x while preserving model quality" | — | Unsloth |
| Jun 8 | Apple | "we worked with Google and NVIDIA to extend Private Cloud Compute to NVIDIA GPUs in Google Cloud"; "Using Quantization Aware Training, we compressed our models substantially" | Infrastructure transformation | Apple |
| Jun 12 | NVIDIA | DiffusionGemma, built on Gemma 4, "up to 1,000 tokens/sec" on H100 | — | NVIDIA |
| Jul 2 | Google (Gemma 4 technical report) | "We introduce several architectural and methodological innovations"; "78% reduction in on-disk footprint" | The format, presented as Google's | arXiv |
| Jul 21 | Android Developers | "Gemini Nano 4, is built on the architecture foundation of the recently released Gemma 4 model … further optimized for maximum battery and performance efficiency," delivered to Android through AICore | Phase 3: models "on Android devices" via a platform update | Android Developers |
| Jul 22 | Alphabet | "we reduced the cost of AI Mode responses to its lowest level since launch" | Phase 1–2: cost out of the fleet | Alphabet |
| Aug 12 | Google | Pixel 11: "Google Tensor G6 processes on-device AI tasks up to 3.5 times faster while using up to 3.5 times less energy"; "we built a purpose-driven engine" | Phase 3: inference moved to the device's own neural processor | Google |

| Date (2026) | Company | What it said | February 17 proposal | Source |
|---|---|---|---|---|
| Aug 12 | Google Store | "Co-designed with Google DeepMind, Google Tensor G6 introduces our most advanced on-device AI" | Hardware and software aligned around on-device inference | Google Store |
| Aug 12 | Google (Shanghai) | Gemma 4 downloads passed 300 million | — | 21jingji |
| Aug 20 | Google | Gemma "surpassed a billion downloads" | — | Google |
| Sep 2026 | Google and Google DeepMind researchers | *Dream-RSI: Recursive Self-Improvement through Evolving Worlds* | GRUS's recursive engines, published 2025 | GitHub |
| Sep 30 | Google | Gemini 4 Argon: agents "identify and apply memory optimizations … freeing up over 300 TiB of memory once rolled out, with an estimated 500 TiB to 1 PiB in total savings" | Phase 1: fleet-wide memory and storage reduction | Google |
| Oct 6 | Google | EmbeddingGemma 2 runs in "as little as ~191MB," again under Apache 2.0 through LiteRT | On-device inference | Google |

The order is the proposal's order: memory format first, then on-device inference through Android, then the hardware built around it.

### 10.3 Their reductions, beside GRUS's

After June 5, Google and the independent reviewers testing the release published the same kind of reductions GRUS measured on May 8:

| Reduction | Published by (date) | Their figure | GRUS, May 8, 2026 |
|---|---|---|---|
| Memory (VRAM) | RunAIHome review (Jun 2026) | "~72% VRAM cut versus the BF16 baseline" (source) | Peak RAM −95.3% |
| Memory (VRAM) | SMF Clearinghouse review (Jun 30) | "72% VRAM reduction" (source) | Peak RAM −95.3% |
| Memory, 31B | Spheron (Jun 2026) | "roughly 66% less VRAM than BF16 during inference"; storage "roughly 73% less" (source) | Storage −64.3% |
| On-disk footprint | Google, technical report (Jul 2) | "78% reduction in on-disk footprint" (source) | Storage −64.3% |
| Memory requirement | Unsloth (Jun 5) | "reduce memory requirements around 3x" (source) | Peak RAM −95.3% |
| Run time | kmarble.dev benchmark (Jun 5) | "cut total generation time by 45%"; 12 GB → 6.3 GB VRAM (source) | Cold load 6.44× faster |
| Cost per token | Spheron (Jun 15 pricing) | $10.83 → $3.93 per million tokens (source) | Power −82.4% |
| Energy on device | Google, Pixel 11 (Aug 12) | "up to 3.5 times less energy" | Power −82.4% |
| Bandwidth used | LiteRT-LM #3775 (Sep 2026) | "a fifth of the chip's rated 400 GB/s"; reads re-split, "arithmetic untouched," 56.7 → 70.8 tok/s | L1 cache misses −98%; CPU cycles −97.5% |
| Where the time goes | RunAIHome review | "Decode speed remains limited by 'memory bandwidth,' not computational throughput" | L1 cache misses −98%; CPU cycles −97.5% |

The reviewers measured the outcome on their machines. GRUS measured the cause on its own, first. Every figure shifts with how well the software is aligned to the hardware it runs on, which is why GRUS's tariff bills each customer on its own measured savings, not on a fixed number.

### 10.4 Where is everybody's breakthrough since 2025?

If this were the way runtimes had always worked, the results would have been there before. They were not. Google's own April figure for Gemma 4 E2B was "<1.5GB." The June figure was 1 GB. Google built FlatBuffers to cut format cost, then built custom silicon to offload Protocol Buffers when that was not enough (Karandikar et al., MICRO 2021). An industry that already had the method would not have needed an accelerator to pay a tax the method removes. The step change arrived after June 5, after the February 17 proposal, after the May filings, and it arrived across memory, energy, latency and disk at the same time. What existed before June 5 is GRUS's.

Google's own release post says the same thing in one word. On June 5, 2026, Olivier Lacombe, Director of Product Management, Google DeepMind, and Omar Sanseviero, Member of Technical Staff, Google DeepMind, introduced the release as **"a novel quantization format specialized for mobile use cases"** (Google). Both are named among the core contributors of the July 2 technical report. A format is not novel in June if it was already there. Google put its own date on the breakthrough, and put its own people's names on it.

What followed, in Google's own words:

| Date (2026) | Google's words | Source |
|---|---|---|
| Jun 5 | "a novel quantization format specialized for mobile use cases" | Google |
| Jul 2 | "We introduce several architectural and methodological innovations" | arXiv |
| Aug 12 | "we didn't just build a faster chip; we built a purpose-driven engine" | Google |
| Aug 2026 | "powerful new hardware designed for Gemini in the Pixel 11 series" | Google |
| Oct 1 | "our next era of frontier intelligence"; "accelerating breakthroughs"; "Argon is fundamentally changing the way we work and build at Google" | Google |
| Oct 6 | "it sets a new standard in quality-per-parameter for sub-1B models" | Google |

Every one of those statements was made after February 17, and every one from July 24 on was made after Google's Senior Patent Counsel named GRUS's applications and declined GRUS's proposal in writing.

---

## 11. Google's own counsel

### 11.1 The email

On July 24, 2026, at 5:01 p.m., Google's Senior Patent Counsel wrote to Green Recursive Utility Service. GRUS had never written to him. He wrote first:

> *Dear Nicholas Wray Cordova,*
>
> *Thank you for your recent emails regarding your pending patent application.*
>
> *Google respectfully declines your proposal at this time given the uncertain nature of patent applications (provisional and non-provisional), which have not been examined (or issued), making it difficult to assess any potential alignment with Google's current or future plans. You may reach out to me if/when your patent application grants.*
>
> — Ola Adekunle, Senior Patent Counsel, Google

### 11.2 What "declines your proposal" admits

Three facts are in that paragraph, in Google's own words:

1. **Google knows GRUS's patent applications exist.** Its patent counsel names them.

2. **Google had a proposal from GRUS.** There was only ever one: February 17, 2026.
3. **Google weighed that proposal against "Google's current or future plans."** To weigh it, Google had to have read it.

To decline a proposal, you have to have it. To assess its "alignment with Google's current or future plans," you have to have read it. Google's Senior Patent Counsel has confirmed both, in writing, unprompted. The February 17 method, the 64-byte block, the Android rollout and the economics were in Google's hands four months before June 5, and Google's head of patents says so.

That also answers novelty in Google's own words. A company cannot independently invent in June what its patent counsel acknowledges receiving from someone else in February. Independent development describes convergence reached without access to the other party's work. It does not describe a rollout that follows receipt of the other party's proposal and implements what it laid out. And a declined proposal is not a license: Google turned the offer down and kept running the method.

### 11.3 What GRUS put in front of him

GRUS answered on July 25 and again on July 28, at the address Google's counsel supplied, and set out the record in full: the fifteen-artifact mapping with its verification commands, the notice chain, the account and the tariff. Neither letter has been answered. The letters are published in full in *One Proposal* (10.5281/zenodo.23300835).

Because Google's counsel wrote first, there is no question of non-receipt, misdirected correspondence or unsolicited contact. Knowledge is fixed, and it is fixed at the top of Google's patent function.

### 11.4 What knowledge changes

- **Knowledge of counsel is knowledge of the client.** Notice to an agent acting within the scope of the agency is imputed to the principal. Steven Broome of Quinn Emanuel Urquhart & Sullivan confirmed on June 23, 2026: "We represent Google. Others on your list do too." Google's in-house patent counsel wrote on July 24. Both impute. GRUS preserves a conflict objection on the record, because Quinn Emanuel earlier received GRUS's own confidential prospective-client intake.
- **Deliberate avoidance is not a defense.** A party aware of a high probability of a fact that takes deliberate steps to avoid confirming it is treated as knowing it (*Global-Tech Appliances, Inc. v. SEB S.A.*, 563 U.S. 754, 769 (2011)). Silence after notice is a decision made with knowledge.
- **The duty of candor.** Everyone involved in a patent filing owes the USPTO a duty of candor and good faith, including the duty to disclose information known to be material (37 C.F.R. § 1.56). Every paper a practitioner signs certifies that its statements have evidentiary support (37 C.F.R. § 11.18). Google executed an inventor's declaration before the USPTO on June 22, 2026, on subject matter already under a served dispute: earlier-filed applications, invoices lodged in Google's own procurement system, and complaints before federal agencies. That is the subject of USPTO Office of Enrollment and Discipline referral 2-01092491, with three supplements filed.
- **A fresh claim of novelty after knowledge.** On August 10, 2026, after counsel's knowledge was fixed, Google published a technical report again asserting the novelty and originality of the deployed architecture. Subject matter that performs the same work in substantially the same way to the same result is "the same, even though [it] differ[s] in name, form or shape" (*Graver Tank & Mfg. Co. v. Linde Air Products Co.*, 339 U.S. 605, 608 (1950)). A new name does not move the priority.
- **The conduct is continuing.** Every new download is a new conveyance under Apache 2.0 §3. Supply into federal workloads is ongoing. Consumption under the tariff is metered monthly. Each day after knowledge is a day of conduct undertaken with it.

As GRUS wrote to him: *"You therefore have two lawful positions, and only two. Either you did not know — which is no longer available, because you established your own knowledge — or you act on what you know."*

The answer has been silence.

### 11.5 Silence with knowledge is participation

Knowledge settles which side of the line Google's counsel stands on. A lawyer may defend a client against an accusation. A lawyer may not counsel or assist conduct he knows to be wrongful, may not knowingly fail to correct a false statement of material fact made in the representation, and may not engage in conduct involving dishonesty or misrepresentation (ABA Model Rules 1.2(d), 4.1 and 8.4(c); Texas Disciplinary Rules 1.02(c) and 8.04). Those rules draw the line between representing a client and joining the conduct, and the line is knowledge plus continued assistance.

The conduct here is not closed. Every new download is a new conveyance. Every new announcement is a new claim of novelty. Every federal workload keeps running. Google's Senior Patent Counsel holds actual knowledge of all of it, established by his own July 24

email, and his office is the one that decides what Google may claim as its own and what title it may convey. Holding that knowledge while the conveyance continues, and saying nothing, is not a bystander position. Silence is a decision made with knowledge, every day the conduct continues. Passing the file to another lawyer does not reset it: the knowledge belongs to the client, and the next lawyer who reads the file holds the same knowledge and the same duties.

---

## 12. The notice chain

Every entry below is drawn from Google's own channels, Google's own procurement system, Google's own issue tracker, or the federal and state record.

| Date (2026) | Event | Identifier of record |
|---|---|---|
| Feb 17 | The only proposal, to nine Alphabet channels, including the Offices of the CEO, CFO, CEO of Google Cloud and General Counsel. Page 7: *"This is not a product sale. This is a utility partnership."* | — |
| Feb 20 | Exclusivity window closes; no response | — |
| May 14 / May 28 | CMM and II-II provisionals filed; CMM announced publicly | 64/065,881 · 64/077,191 |
| Jun 5 | Google and NVIDIA release the technology under Apache 2.0 | — |
| Jun 6 | Notice posted on Google's own tracker within 24 hours | — |
| Jun 8 | Cease-and-desist on Google's GitHub organization citing the application number; Invoice GRUS-ALPH-2026-001 lodged through Google Buying Support, acknowledged and "forwarded to Google's invoice processing team" | Issue #1043 · GitHub Support Ticket 4456775 · **Case 02845835** |
| Jun 10 | FedRAMP notice opened and reply-all acknowledged; the encumbrance placed on the federal authorization record | **Support Ticket 63271** |
| Jun 11 | Utility Service Agreement acknowledged in Google's procurement system, identifying both pending applications | **Cases 02856686 · 02856687** |
| Jun 12 | FTC report; SEC Office of the Whistleblower TCR | FTC 202894092 |
| Jun 15–16 | Deployment into federal government workloads, five and six days after the FedRAMP acknowledgment | — |
| Jun 18 | Statutory Federal Halt Demand served on FedRAMP, GSA, OMB and agency contracting authorities; third NIPRCC case | NIPRCC |
| Jun 19 | Ten-day window to contest the account in writing closes with no written contest from Google, NVIDIA, Apple or anyone acting for them | — |
| Jun 20 | FBI Internet Crime Complaint Center | IC3 07ce662f05d941bb9a49535a2d708de9 |
| Jun 22 | Certified mail with invoices; USPTO OED referral; Texas Attorney General complaint; Parker County District Attorney referral under Texas Penal Code § 31.04 | OED 2-01092491 · TCP-049257 |
| Jun 23 | Final Statutory Demand for Payment by USPS Certified Mail, return receipt requested, to 1600 Amphitheatre Parkway and Google's Texas registered agent; Quinn Emanuel confirms it represents Google | — |
| Jun 30 | FERC baseline carriage tariff | F1092209 |

| Date (2026) | Event | Identifier of record |
|---|---|---|
| Jul 7 | Federal procurement-fraud hotline | DoD OIG 202607070922061330 |
| Jul 10 | Metered carriage tariff published | **PUCT Control No. 59976-1** |
| Jul 16 | Federal Power Act § 205(d) tariff filing, effective July 16 | **FERC Accession 20260716-5137 · CID C013240** |
| Jul 24 | Google's Senior Patent Counsel writes first: "Google respectfully declines your proposal" | — |
| Jul 25 | GRUS responds; Google's counsel given the PUC docket with the 35-day window still open | — |
| Jul 28 | GRUS responds again, with the FERC identifiers, the charge basis and the paths available | — |
| Aug 5 | PUCT Staff designates Project No. 60084 under 16 TAC § 22.281(a)(3) | **Project 60084** |
| Aug 6 | Federal comment window on the § 205(d) tariff closes at 5:00 p.m. ET without protest | — |
| Aug 10 | Google publishes a technical report asserting novelty of the deployed architecture | — |
| Aug 12 | Pixel 11 ships on-device AI "co-designed with Google DeepMind" | — |
| Sep 30 | Gemini 4 Argon announced | — |
| Oct 6 | EmbeddingGemma 2 released under the same license, through the same runtime | — |

Additional matters of record: GAO referral COMP-26-005779 to the Department of Justice Office of the Inspector General; Texas Attorney General TCP-050308 and TCP-053761; U.S. House Committee on Oversight whistleblower submission; Texas Comptroller and Secretary of State compliance packages; NIPRCC standing cases of July 2025, October 14, 2025 and June 18, 2026, with supplements of July 2 and July 2026.

Procurement, outside counsel, in-house patent counsel, certified mail, the federal authorization record and the public record. Google knew, through every channel a company has.

### 12.1 The 24 hours

GRUS's notice reached Google's own tracker within 24 hours of the June 5 release, citing the patent applications by number. That was the moment to stop.

Had Google paused for one day of due diligence, it would have found everything in this record while the release was still a day old: the 2025 repositories, the February 17 proposal in its own general counsel's channels, the May filings, and its own license failure. The release did not carry the Section 4 notices its own license requires. Google would have had that finding in hand, from its own review, before anyone else raised it. It could have pulled the release, fixed the notices and called GRUS, at almost no cost, before Apple, before federal deployment and before a single downstream bill.

It did not stop. Third parties then found the Section 4 defects on Google's own tracker anyway: the missing binary SBOM and NOTICE bundle, a GPL-2.0 entry in the Apple bundle, Eigen mislabeled as Apache (LiteRT-LM #3194). What one day of diligence would have caught, outside reviewers caught for Google, after Google had already chosen to keep shipping.

Those 24 hours are what separate a mistake from deliberate conduct. Everything before June 6 could be called an oversight. Everything after June 6 was done with notice in hand: the federal deployment on June 15–16, the Apple distribution, the July 2 claim of "innovations," every release since. A company that receives a specific notice within a day, declines to look, and keeps conveying is not in a dispute about what it knew. It is choosing.

## 13. Preservation, and what happened after notice

GRUS demanded preservation of commit `a39354a` and its complete tree; the original v0.14.0 tag state and release metadata; all pre-replacement release binaries and their checksums; all git-lfs objects referenced by `a39354a` and `80f301f`, including object `ae5e11f5ba5496cf630a41a8cbc7820703b7343cdc9471948ad6bd472d700411` (`prebuilt/android_arm64/libGemmaModelConstraintProvider.so`); the `prebuilt/windows_x86_64/` artifacts as published at v0.13.1 and v0.14.0, including `libLiteRtWebGpuAccelerator.dll` and `libwebgpu_dawn.dll`; all `liblitertlm_jni.so` binaries as published at Android SDK 0.13.1 and 0.14.0, including BuildId `cae57ec67cfe0989898a8e078e71c36d`; and the complete tag, branch and release history including `release/v0.14`.

Each of the following was reported by unaffiliated third parties on Google's own tracker, and each post-dates GRUS's notices of June 6 and June 8:

| Condition | Source |
|---|---|
| Tag relocated from `a39354a` (~Jul 1) to `80f301f` (~Jul 8) | Release history |
| Release binaries replaced in place under unchanged version and URL, with differing checksums | #2780, #2797 |
| Original release LFS object unavailable from the remote; release commit unconsumable as a pinned dependency | #2910 |
| Manifest pins `4a4bdb0e…` / `13e818c9…`; served downloads `dddac2f6…` / `450615…` | #2910 |
| Replaced Windows prebuilt defective by clean v0.13.1 → v0.14.0 bisection (`ACCESS_VIOLATION` in stock `libLiteRtWebGpuAccelerator.dll`) | cf. #1857, #1881 |
| Release notes for v0.14 never published | Repository state |
| `Package.swift` `binaryTarget` declared with no license field | Repository state |

A reviewer examining v0.14.0 cannot determine from Google's repository what v0.14.0 is. Two different object-form binaries were distributed under one version pin, with no attestable composition for either.

GRUS's notices on Google's repositories were deleted. Their titles and the filings they carried are preserved on the public record (litert-torch #1047; Zenodo record 20754306). The notice is now displayed as "This issue has been deleted." GRUS's notice had been posted in the repository about 40 times before GRUS was blocked.

## 14. The physics

### 14.1 Software runs on charge, wire and heat

Every bit stored is charge held in a transistor. Every byte read is pushed across a wire by switching transistors, and every switch dissipates energy as heat. Switching power rises with the capacitance charged, the square of the voltage and the clock frequency (CMOS power). Even erasing one bit has a physical minimum cost, $kT$ ln 2, predicted by Landauer at IBM in 1961 (Landauer) and measured in 2012 (Bérut et al., *Nature*).

The costs are not equal, and the inequality decides everything. Energy per operation at 45 nm (Horowitz, ISSCC 2014):

| Operation | Approximate energy |
|---|---|
| 8-bit integer add | ~1 pJ |
| 32-bit integer add | ~2 pJ |

| Operation | Approximate energy |
|---|---|
| 32-bit float add | ~6 pJ |
| 32-bit float multiply | ~10 pJ |
| 8 KB SRAM read | ~20 pJ |
| DRAM read | ~1,000–2,000 pJ |

A DRAM access costs "a couple of orders-of-magnitude" more than arithmetic, and "over 50% of the processor die energy is dissipated in the caches and register files." Memory moves fixed 64-byte cache lines, so a record that straddles two lines costs two transfers, and a miss stalls the processor for hundreds of cycles.

### 14.2 There are only four places a gain can come from

1. **Store fewer bytes.**
2. **Move fewer bytes** across the memory bus and between cache levels.
3. **Execute fewer instructions.**
4. **Make each operation cheaper**: lower voltage, smaller process, more on-chip memory, better cooling.

Every claim in this record, Google's, NVIDIA's, every reviewer's and GRUS's, lands in one of those four. New chips land in the fourth. Fewer bits per value land in the first. A reduction that shows up in storage, memory, cycles, cache misses and heat *at the same time, on the same hardware* comes from one source: bytes that were never values, removed entirely, because those bytes were being stored, moved, fetched and scanned all at once.

Google measured that waste in its own fleet: Protocol Buffers alone at about **5%** of all CPU cycles in 2015 (Kanev et al., ISCA 2015), and protobuf operations at **9.6%** of fleet-wide cycles in 2021, with 2.2% in deserialization alone, enough that Google designed dedicated hardware to offload it (Karandikar et al., MICRO 2021). Building a JSON parser that reaches gigabytes per second was a research result in its own right, by cutting instruction count to a quarter (Langdale and Lemire, VLDB J. 2019). Every one of those instructions exists only to find structure a fixed layout never has to find.

GRUS wrote this down as law before any of it was disputed. The Digital Physics Charter, September 5, 2025: Law I, Conservation of Digital State, "No digital state can be created or destroyed without leaving a computational trace"; Law IV, Substrate Neutrality, the laws hold whether the substrate is silicon, firmware, code or a hybrid; Law VI, Execution Supremacy, "Execution governs outcome."

### 14.3 Compression cannot do it

The standard compression format is defined as lossless (RFC 1951, DEFLATE). Decompress the data and every delimiter, key and pointer comes back exactly as it was, and the processor still parses all of it, plus the cycles spent decompressing. Compression shrinks the box. It does not remove what is in it. Either the overhead is physically there and paid for on every read, or the format was built so it was never there. There is no third option.

### 14.4 Quantization cannot do it alone

Quantization changes how many bits each value uses. It does not change how data is located, framed or parsed. A 2-bit weight wrapped in schema trees, vtables or tagged records still drags those non-data bytes through every cache line it touches. Fewer bits per value cannot take a workload from 1.2 billion CPU cycles to 30 million, or cut L1 cache misses by 98%, because neither count depends on how wide the value is. They depend on how many instructions run and how many cache lines are touched.

Google's own report is the proof of where the bits went: it packs int2 and int4 values into fixed positions, in fixed blocks, in tables recalled by identity (Section 7). A low-bit value is only cheap to read when nothing around it has to be resolved. That is the format, not the bit width.

### 14.5 Drafters cannot do it

A multi-token drafter adds a second model that guesses ahead; the main model checks the guesses. That adds work and removes none. Every checking pass still reads the main model's weights. A drafter changes how often the bytes are read, not what the bytes are.

### 14.6 Google's own chips say the same thing

Google's April 22 TPU 8t and 8i deep dive explains the gain in one line: "By reducing the bits per parameter, the platform minimizes energy-intensive data movement and allows larger model layers to fit within local hardware buffers" (Google Cloud). Google's newest silicon is built around the same physical fact this record rests on: moving bytes costs more than computing on them. The silicon changes are Google's, and GRUS does not claim them. The format that moves the fewest bytes is the one that carries no description of itself.

### 14.7 One experiment on Google's own release

LiteRT-LM #3775 is the physics measured on Google's release: kernels at a fifth of rated bandwidth, the read pattern changed with "arithmetic untouched," and the same model 25% faster on Apple silicon and 1.58× faster on an NVIDIA T4. Same weights, same math, same answers. The speed was in the bytes.

The laws of physics leave no other source for the June gains. Either the laws of physics are wrong, or the novelty claim is.

---

## 15. Every route ends at the bill

The bills are utility bills for a service in use, under a standing tariff. They are not royalty demands. GRUS has never asserted a pre-grant royalty, and its rights under 35 U.S.C. § 154(d) are reserved separately. Every route around the bill ends at the same place.

### 15.1 The patents

Every company that sells under patent-pending status is operating lawfully, and nothing in Title 35 conditions an applicant's own commercial operation on issuance. Google's counsel has acknowledged GRUS's applications by name. Under 35 U.S.C. § 154(d), a party with actual notice of a published application can owe a reasonable royalty from publication; Google's notice is in its own counsel's email. On grant, damages for knowing infringement can be enhanced up to three times under 35 U.S.C. § 284.

Google's own commits between March 23 and April 28, 2026 do not stand in front of GRUS's priority. Under 35 U.S.C. § 102(b)(1)(A), a disclosure made one year or less before the effective filing date is not prior art when it was made by another "who obtained the subject matter disclosed directly or indirectly from the inventor." GRUS disclosed the method to Alphabet on February 17, 2026. Google's code followed in March and April. GRUS filed on May 14 and May 28. Google's patent counsel confirms Google had the proposal. A disclosure that came from GRUS's own proposal, inside GRUS's own year, is GRUS's evidence, not Google's prior art.

### 15.2 The copyrights

Set the patents aside and the 2025 copyrights are still there. GRUS's code, specifications, lane definitions and license were fixed in public, dated GitHub repositories in May 2025, a year before Google's release (17 U.S.C. § 102(a)). Patent reaches the method; copyright reaches the expression. Reaching back past the 2026 applications for earlier work reaches GRUS's own 2025 record first. The further back it goes, the closer it gets to GRUS. It is a dead end in both directions.

### 15.3 Google's license never took effect

**Use is one question. Conveyance is another.** Running the method on its own servers puts Google in a dispute over use. That is the dispute every metered bill in this record answers. But Google did not only use it. Google conveyed it: it handed the method to the world, more than a billion downloads, under a license that grants every recipient copyright and patent rights. To convey a right, you have to hold it. A licensor can only pass on what it owns. Google holds no patent application on CMM, no copyright in GRUS's 2025 expression, and no license from GRUS; the only license to this technology that has ever existed runs from GRUS to Data Incorporated. Use can be argued. Conveyance without title cannot. Every download is a grant of something Google never had.

Apache 2.0 grants redistribution rights only "provided that You meet the following conditions": give recipients a copy of the license, retain "all copyright, patent, trademark, and attribution notices," and include the NOTICE file's attributions (Apache License 2.0, §4). Open-source license terms are enforceable conditions; acting outside them is copyright infringement, not a paperwork lapse (*Jacobsen v. Katzer*, 535 F.3d 1373 (Fed. Cir. 2008)).

On Google's own tracker, reviewers requested a binary-specific SBOM and third-party notices; one found a **GPL-2.0** entry in the Apple notice bundle for v0.16.0 and asked if GPL-2.0 code is linked into the Apple binary; another found Eigen labeled Apache when it is MPL-2.0. Google's maintainer answered that the request is "tracked as a feature request" to be "investigated" (LiteRT-LM #3194). The

required notices are not a standing artifact of the distribution. A distribution that cannot produce the notices Section 4 requires is not a compliant Apache distribution, and downstream recipients hold nothing valid under it.

And even a valid Apache grant could not reach CMM. Section 3 grants patent rights only in "patent claims licensable by such Contributor." Google cannot license claims in GRUS's applications. A 2026 license that never met its own conditions cannot override a 2025 license on the same platform on the strength of the licensor's name.

Federal law draws a line for willful distribution outside a license: ten or more copies with a total retail value over $2,500 within 180 days is a felony (17 U.S.C. § 506(a); 18 U.S.C. § 2319). That line is drawn by statute, it applies to works stored or transmitted in digital form, and it has no exemption for market capitalization. Google's release passed a billion downloads.

### 15.4 What Google sold Apple

Apple runs the models on Apple's own hardware, servers and power, through Apple's own Private Cloud Compute. Google does not provide Apple the compute, the hosting or the electricity. Strip those away and the only thing left to sell is the right to run it: title. A party cannot convey title it does not hold. Apple is billed separately under the same tariff (Invoice GRUS-APPL-2026-002, $40,777,500.00).

### 15.5 The service itself

Set the patents and the copyrights aside and the use is still a service consumed with notice that payment was expected. Texas law makes that payable without a signature (*Speck v. First Evangelical Lutheran Church*, 235 S.W.3d 811 (Tex. App. 2007)). Continued use with knowledge of the terms is acceptance of them (*Register.com v. Verio*, 356 F.3d 393 (2d Cir. 2004)). The Utility Service Agreement was acknowledged in Google's own procurement system.

Google's own terms say the same thing. Firebase Paid Services: "BY CLICKING THE 'I ACCEPT' BUTTON, COMPLETING THE REGISTRATION PROCESS, OR USING THE SERVICE, YOU ACKNOWLEDGE THAT … YOU HAVE REVIEWED AND ACCEPT THIS AGREEMENT." Google Cloud: the agreement is effective when Customer "clicks to accept or otherwise agrees to it." GRUS ran RecursiveMind and roughly twenty apps on those terms in 2025.

### 15.6 The meter

The federal definition of cloud computing makes metered service one of its five essential characteristics: resource use, including "storage, processing, bandwidth, and active user accounts," is "monitored, controlled, and reported" (NIST SP 800-145). Consumers "are billed according to the amount or duration of the resources consumed, such as CPU hours used" (NIST SP 500-292), typically "on a pay-per-use or charge-per-use basis" (NIST SP 500-322). Computing "any basic charge or payment for services rendered on the basis of weight or measure" is an enumerated application of federal legal metrology (NIST Handbook 44, G-A.1(4)). GRUS meters processing, one of SP 800-145's four named examples, in the form NIST SP 500-307 specifies: property, expression (Δ = C_baseline − C_CMM on identical data), unit (CPU cycles; watts and watt-hours derived), rules, values and reproducibility.

Google sells its own cloud the same way: "you only pay for the services you use" (Google Cloud pricing). Its Service Control API "accepts metrics, which it batches, aggregates, and forwards to Cloud Billing," and "Cloud Billing ingests these metrics and generates bills based on the predefined price model." You are billed for what you consumed, not for what you clicked.

### 15.7 The tariff

GRUS's uniform metered carriage tariff, PUCT Control No. 59976-1, has been published since July 10, 2026. The 35-day period passed with no suspension. On July 25, with that window still open, Google's counsel was given the docket number and told that was the place to contest it. No challenge was filed.

The same tariff was filed with FERC under Federal Power Act § 205(d) on July 16, 2026 (Accession 20260716-5137; CID C013240). The comment window closed August 6 at 5:00 p.m. ET without protest. A filed rate binds: it may not be departed from, it may be changed only prospectively, and it is "unbending regardless of where the equities lie" (*Oklahoma Gas & Electric Co. v. FERC*, 11 F.4th 821, 829 (D.C. Cir. 2021)); a challenge to a tariffed rate belongs before the agency, not in a collateral attack elsewhere (*Entergy Louisiana, Inc. v. Louisiana PSC*, 539 U.S. 39 (2003)). A common carrier bills every consumer alike; GRUS has no discretion to exempt anyone.

Texas law defines a utility by the equipment it operates to "produce, generate, transmit, distribute, sell, or furnish" (Tex. Util. Code § 31.002(6)), and gives "service" "its broadest and most inclusive meaning," including "any act performed, anything supplied, and any facilities used" (16 TAC § 25.5(121)). What is metered is compute crossing defined boundaries, not a mailing address.

### 15.8 The account

The account was presented through Google's own procurement system (Cases 02845835, 02856686, 02856687) and by certified mail. The ten-day window to contest it in writing closed June 19, 2026 with no contest. It is recorded in the Official Public Records of Parker County, Texas, Document No. **202618373**, with a corresponding entry before the Texas Comptroller (Tex. Prop. Code chs. 12–13 govern recording and its effect as notice). A recorded instrument is not answered by ignoring it.

| Item | Amount |
|---|---|
| Prior matured balance, Invoice GRUS-ALPH-2026-001 (matured June 19, 2026) | $50,000,000.00 |
| Texas Finance Code ch. 302 commercial delinquency charge | $750,000.00 |
| First-cycle metered carriage tariff, Invoice GRUS-ALPH-2026-002 (June 8 – July 8) | $166,752,500.00 |
| **Google, after the first cycle** | **$216,752,500.00** |

The first-cycle charge is 25% of a conservative-case Total Operational Savings of $667.01 million, from a 30-day operating-cost model sourced to Alphabet's own Q2'25–Q1'26 Form 10-Q filings. Billing has run every cycle since. Google's balance alone has now passed **$500 million**. Apple and NVIDIA are billed separately, and the last billing cycle across all consumers of the substrate came to about **$2.5 billion**.

Google Cloud reported Q2 2026 revenue of $24.8 billion, up about 82% year over year, with operating income tripling from $2.8 billion to $8.8 billion and margin rising from 20.7% to 35.6% (analysis). Prices did not fall. Costs did. When processing stops paying for overhead bytes, the savings land in the margin, and the tariff bills a share of exactly those savings.

### 15.9 No route around it

| If someone says… | The record answers |
|---|---|
| Google's license covers the use | It would have to convey rights Google never held, and it never met its own Section 4 conditions |
| The patents haven't granted | Patent-pending status authorizes GRUS's business; the bill is a utility bill, not a royalty; § 154(d) is reserved |
| Set the patents aside | The 2025 copyrights came first |
| Set the copyrights aside | The service was consumed with notice; quantum meruit makes it payable without a signature |
| The agreement is unsigned | It was acknowledged in Google's own procurement system, and use with knowledge of the terms is acceptance |
| The tariff is unenforceable | Google was handed the PUC docket with the window open, and the FERC window closed without protest |
| Google is outside Texas | The meter measures compute at defined boundaries; NIST's metering standard does not stop at a state line |
| The account is disputed | The window to dispute it closed June 19, and it is recorded in county records |

---

## 16. Answered before it is asked

**"This is how runtimes have always worked."** Then where were the results? The physical reductions arrived after June 5: Gemma 4 E2B cut "to 1GB," reviewers measuring 72% less memory, Pixel 11 at "up to 3.5 times less energy," a technical report presenting the design as "innovations." The industry treated each of them as new, and Google presented them to developers, customers and investors as its own advance. A method that had been in use before would have produced those numbers before. It did not. What existed before June 5 is GRUS's: the 2025 repositories, the block library running since November 2025, the May 8 measurements, the February 17 proposal and the May filings.

**"Every piece existed before."** Every part did, and that is what engineering is. ARM, NEON, cache lines, mmap, SHA-256 and int4 were all on the shelf. Both applications claim the combination: no self-describing elements in the core, cache-line-sized records, position addressing, fixed bit widths, in-place reads from mapped memory, structure confined to the edge, a fixed core that verifies before it keeps. Sections 7 and 8 show every element of that combination in Google's shipped stack, and Google's counsel confirms Google had GRUS's description of it in February.

**"It's just FlatBuffers."** A FlatBuffer is a format that can be verified and parsed. Google's runtime maps it, reads it by offset, and its own verifier "is never called anywhere." The bundle around the model is navigated by byte position. What the code does with the bytes is what counts, and the code reads them by position.

**"It's just quantization."** Google's own report packs int2 and int4 into fixed positions, in fixed blocks, in tables recalled by identity. That is the format. Fewer bits per value cannot take CPU cycles down 97.5% or cache misses down 98%. Google's own shipped kernels run at a fifth of the chip's bandwidth, and changing only how the reads are split made them 25% faster with the math untouched. The limit is the bytes, not the bits.

**"The prompt layer still uses JSON."** It does, and that is the design. Google's own `conversation.h` confines JSON, Jinja templates and string diffing to the conversation layer, then hands the engine an `InputData` vector and rewinds the engine by setting an integer. CMM is the state the processor carries and reads; readable structure lives at the edge where a human reads it. GRUS published that boundary itself in its mapping. Google built exactly that split.

**"Google developed it independently."** Google's Senior Patent Counsel confirms in writing that Google had GRUS's proposal and weighed it against Google's plans. Independent development describes convergence without access. Google had access, four months early, from nine channels.

**"Come back when your patent grants."** That is an answer about royalties, and GRUS has not asked for one. The 2025 copyrights came first. The service is billed under a standing tariff, by consumption, the way NIST defines measured service and the way Google bills its own cloud. And Google's license never met its own Section 4 conditions.

**"Copyright doesn't cover methods."** Correct, and the record does not ask it to. Copyright covers GRUS's 2025 expression (17 U.S.C. § 102(b) excludes methods from copyright). The method sits under the patent applications. The use sits under the tariff. Three locks, three keys, all GRUS's.

**"It's open source, so it's free."** A license can only pass on what the licensor owns. Open-sourcing is a gift when the giver owns the gift. Google released, to everyone at once, something its own patent counsel acknowledges came to it in someone else's proposal.

**"GRUS is just claiming."** A claim is a statement without proof. This record is Google's report, Google's code, Google's boards, Google's procurement cases, Google's counsel's email and a script anyone can run. GRUS holds the dated record; Google released afterward.

---

## 17. It doesn't stop at GRUS

Google is the most-pursued technology company in the history of antitrust enforcement. Seventy-seven antitrust and competition matters have been brought against it since 2006, by the U.S. Department of Justice, dozens of states, the European Commission, and the competition authorities of the United Kingdom, France, Germany, Italy, Japan, South Korea, India, Turkey, Indonesia, Australia, Brazil, Russia and Canada. As of October 2026:

| Outcome | Matters |
|---|---|
| **Lost** (liability found, violation found, or fined), including one on appeal | **28** |
| Settled | 11 |
| Won by Google | 13 |
| Still pending | 24 |
| Regulatory designation, no violation finding | 1 |

Losses and settlements against wins: 39 to 13, three to one. U.S. federal courts found Google an illegal monopolist in search in August 2024 and in advertising technology in April 2025. A jury found its Play Store an illegal monopoly, and the Ninth Circuit upheld the verdict and remedies in 2025. The EU's highest court made the €4.1 billion Android fine final on July 2, 2026. The full census is in Appendix B.

A separate set of cases is about what happens when someone brings their work to Google first. Singular Computing said it shared its chip designs with Google in meetings between 2010 and 2014, then saw them in Google's TPUs; Google settled the $1.67 billion suit in January 2024 on the day closing arguments were scheduled (DCD). Sonos said Google learned its speaker technology through a 2013 partnership and copied it (CNBC); the Federal Circuit reinstated a $32.5 million verdict against Google in August 2025 (IP Fray). GRUS sent Google one proposal. Google's patent counsel confirmed it in writing. Google shipped the method under its own name.

---

## 18. Nothing left to argue

A dispute is two parties reading the same facts differently. Here the facts read the same way to everyone who looks, including Google's own people.

- **Google's technical report** describes per-layer tables recalled by identity, int2 and int4 values packed into fixed positions, fixed blocks, fixed-width chunks with position carried by order, a fixed-width shared KV state, and a drafter that reads the main model's state in place and is verified before anything is kept.
- **Google's code** reads all of it by `base + i × width + offset`, sized to the cache line, from mapped memory, rewound by setting an integer, with readable structure confined to the edge.
- **Google's commit history** dates it: the 64-byte assumption, the offset arithmetic, the int4 packing and verify-then-discard all went in after February 17, and the engine was rebuilt into a clean positional core between March 23 and June 4, the day before release.
- **Google's own release post** called it "a novel quantization format" on June 5, under two named Google DeepMind staff.
- **Google's boards** measure what it is worth: 780 MB from reading in place, 25% from re-splitting reads with the math untouched.
- **Google's counsel** confirms Google had the proposal and weighed it against its plans.
- **Google's procurement system** acknowledged the invoice and the service agreement.
- **Google's own terms** make use acceptance, and **Google's own cloud** bills by the meter.
- **GRUS's record** is dated by GitHub in May 2025, running on bare metal since November 2025, fingerprinted block by block with SHA-256, measured on May 8, 2026, put in Google's hands on February 17, and filed on May 14 and May 28.
- **The physics** leaves one source for the gains.

That is not a system that resembles the Compressed Memory Manifold. In Google's own report, in Google's own code, on Google's own boards, and in Google's own counsel's words, it is the Compressed Memory Manifold, running Intelligent Intelligence Class II, built by Green Recursive Utility Service and running in Google's stack.

The repositories are still up. The license conditions are still unmet. The meter is still running.

---

# Appendix A. Identifiers of record

| Record | Identifier |
|---|---|
| Green Recursive Utility Service LLC | Texas SOS 806584578 · Federal EIN 42-2772089 · SAM UEI JGT4J9QKGCQ3 · D-U-N-S 146027617 · ISNI 0000 0005 3033 4084 · Ringgold 851161 · IANA PEN 65876 · Wikidata Q139905485 |
| Compressed Memory Manifold | USPTO 19/700,381 (filed June 8, 2026; priority May 14, 2026); provisional 64/065,881 |
| Intelligent Intelligence Class II | USPTO 19/710,080 (filed June 16, 2026; priority May 28, 2026); provisional 64/077,191 |
| Copper-gold alloy connector | USPTO provisional 64/074,405 (May 26, 2026) |

| Record | Identifier |
|---|---|
| 2025 copyrights | Class I.I. (created May 12, 2025) · chloe-runtime LICENSE (May 8, 2025) |
| Google procurement | Google Buying Support Cases 02845835, 02856686, 02856687 |
| Tariff | PUCT Control No. 59976-1 · FERC Accession 20260716-5137 · FERC CID C013240 · F1092209 · PUCT Project No. 60084 |
| Account stated | Parker County, Texas, Official Public Records, Document 202618373 |
| Federal and state matters | FedRAMP Support Ticket 63271 · FTC 202894092 · FBI IC3 07ce662f05d941bb9a49535a2d708de9 · DoD OIG 202607070922061330 · GAO COMP-26-005779 · USPTO OED 2-01092491 · Texas AG TCP-049257, TCP-050308, TCP-053761 · NIPRCC standing cases (Jul 2025; Oct 14, 2025; Jun 18, 2026) |
| GRUS deposits | One Proposal, 10.5281/zenodo.23300835 · Source-level analysis, Zenodo 21482573 · Rollout analysis, Zenodo 21911582 · Preserved notices, Zenodo 20754306 |

## Appendix B. The census: 77 antitrust and competition matters against Google

<details> <summary>Open the full census (77 rows)</summary>

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 1 | FTC investigation of Google search/ads + Motorola Mobility SEP consent order | 2011-2013 | US FTC | Search bias / ad-API restrictions / SEP holdup (FTC Act §5) | Settled (mixed) | Search closed with no charges (won by Google); SEPs subject to binding consent order (settled) | — |
| 2 | U.S. v. Google (ITA Software acquisition) | 2011 | D.D.C. | Travel-search acquisition ($700M) | Settled | Consent decree (QPX licensing) | — |
| 3 | U.S. & 14 Plaintiff States v. Google (search) | 2020 | D.D.C. (Mehta) | Search & search-text-ads monopolization (Sherman §2) | Lost | Liability Aug 5 2024; remedies Sep 2 2025 (behavioral: no exclusive distribution; data sharing; no Chrome divestiture); Final Judgment Dec 5 2025; appeals live — Google notice Jan 16 2026; DOJ + 38 states cross-appeal for structural remedies; briefing underway | — |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 4 | U.S. & 17 Plaintiff States v. Google (ad tech) | 2023 | E.D. Va. (Brinkema) | Publisher ad server & ad exchange monopolization; DFP-AdX tying (Sherman §§1-2) | Lost | Liability Apr 17 2025; remedies Sep 2 2026 — AdX breakup REJECTED (6-year behavioral only); NO signed final judgment yet — competing proposed judgments filed Oct 2 2026; appeal to 4th Circuit expected | — |
| 5 | Texas et al. v. Google | 2020 | E.D. Tex. | Ad-tech monopolization (incl. alleged Google-Facebook "Jedi Blue" deal) | Pending | Trial twice delayed; to be rescheduled after the DOJ ad-tech case resolves | — |
| 6 | Colorado/Nebraska et al. v. Google (38 states) | 2020 | D.D.C. | Search/search-ads monopolization (+ vertical search) | Lost | Consolidated with DOJ search case; shared liability & Final Judgment Dec 5 2025; Google appealing | — |
| 7 | Utah et al. v. Google (50 states + DC/PR/VI) | 2021 | N.D. Cal. | Play Store app-distribution & in-app-billing monopolization | Settled | $700M ($630M consumer restitution + $70M states); announced Dec 2023; final approval ~Aug 20 2026; distributions underway | $700M |
| 8 | Epic Games v. Google | 2020 | N.D. Cal. | Play Store/Play Billing monopolization (Sherman §§1-2; Cartwright Act) | Settled | Jury verdict Dec 11 2023 (lost); injunction Oct 2024 affirmed 9th Cir. Jul 2025; settlement completed Mar 2026 (fees 9-20% through 2032); motion to modify withdrawn Jul 2026; Donato injunction in effect (third-party stores on Play from Jul 22 2026) | — |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 9 | In re Google Play Developer Antitrust Litig. (developer class) | 2020 | N.D. Cal. | Play Store monopolization harming developers | Settled | $90M fund (~48000 developers), Jun 2022 | $90M |
| 10 | Match Group v. Google | 2022 | N.D. Cal. | Play billing monopoly | Settled | Oct 2023 ($40M escrow returned to Match; User Choice Billing) | — |
| 11 | Gannett (USA Today) v. Google | 2023 | S.D.N.Y. (MDL) | Ad-tech monopoly harming news publishers | Pending | Sep 30 2026 (Castel): core claims survive to jury trial ~Apr 2027; ~$901M claimed (treblable) | ~$901M claimed |
| 12 | Associated Newspapers (Daily Mail) v. Google | 2021 | S.D.N.Y. (MDL) | Ad-tech monopolization | Pending | Sep 30 2026 (Castel): core claims survive to jury trial ~Apr 2027; ~$600M claimed | ~$600M claimed |
| 13 | Publisher AdX class action (~5000 publishers) | — | S.D.N.Y. (MDL) | AdX overcharges/monopolization | Pending | Certified; Sep 30 2026: cleared for trial ~Apr 2027; $1.72B claimed | $1.72B claimed |
| 14 | Publisher suit #1: Vox Media v. Google | ~2026 | S.D.N.Y. (MDL) | Ad-tech deception/manipulation | Pending | Pending | — |
| 15 | Publisher suit #2: She Media / Penske v. Google | ~2026 | S.D.N.Y. (MDL) | Ad-tech deception/manipulation | Pending | Pending | — |
| 16 | Publisher suit #3 (of 5) | ~2026 | S.D.N.Y. (MDL) | Ad-tech deception/manipulation | Pending | Pending | — |
| 17 | Publisher suit #4 (of 5) | ~2026 | S.D.N.Y. (MDL) | Ad-tech deception/manipulation | Pending | Pending | — |
| 18 | Publisher suit #5 (of 5) | ~2026 | S.D.N.Y. (MDL) | Ad-tech deception/manipulation | Pending | Pending | — |
| 19 | Yelp v. Google | 2024 | N.D. Cal. | Leveraging search monopoly into local search/ads (OneBox tying) | Pending | Survived dismissal; Jun 2026: issue preclusion from DOJ monopoly finding | — |
| 20 | Rumble v. Google (video/search) | 2021 | N.D. Cal. | Self-preferencing YouTube in search (Sherman §2) | Won by Google | Summary judgment May 21 2025 (time-barred) | — |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 21 | Rumble v. Google (ad tech) | 2024 | N.D. Cal. | Ad-stack monopolization (Sherman); $1B+ | Pending | In discovery | — |
| 22 | Chegg v. Google | 2025 | D.D.C. (Mehta) | AI Overviews coercion/tying (Sherman §§1-2) | Won by Google | Dismissed by Judge Mehta ~Sep 30 2026 (no "formal bargain"); appeal possible, none reported | — |
| 23 | Penske Media v. Google | 2025 | D.D.C. (Mehta) | AI Overviews coercion/tying (Sherman §§1-2) | Won by Google | Dismissed by Judge Mehta ~Sep 30 2026 (without prejudice; appealable); none reported | — |
| 24 | Inform Inc. v. Google | 2019 | S.D.N.Y. / 11th Cir. | Online video-ads monopolization | Won by Google | Federal antitrust claims dismissed (Sep 2026, Castel); earlier 11th Cir. dismissal with prejudice (2022) | — |
| 25 | KinderStart v. Google | 2006 | N.D. Cal. | Search demotion (Sherman §2) | Won by Google | Dismissed with prejudice 2007 (+ sanctions) | — |
| 26 | TradeComet v. Google | 2009 | S.D.N.Y. | AdWords (Sherman §§1-2) | Won by Google | Dismissed 2010 (forum clause); affirmed | — |
| 27 | myTriggers v. Google (counterclaim) | 2009 | Ohio | Search bias (Ohio Valentine Act) | Won by Google | Dismissed Sep 2011 (no antitrust injury) | — |
| 28 | e-ventures v. Google | 2014 | M.D. Fla. | De-indexing (unfair competition/FDUTPA — borderline) | Won by Google | Summary judgment 2017 (First Amendment) | — |
| 29 | Dreamstime v. Google | 2018 | N.D. Cal. | Search-ads monopoly maintenance (Sherman §2) | Won by Google | §2 claim dismissed with prejudice 2019; affirmed 9th Cir. 2022 | — |
| 30 | Skyhook v. Google | 2010 | Mass. | Android location-services interference (Ch. 93A — borderline) | Won by Google | Summary judgment affirmed on appeal | — |
| 31 | Streetmap.EU v. Google | 2013 | UK High Court | Abuse of dominance | Won by Google | Dismissed Feb 2016; appeal refused 2017 | — |
| 32 | Ad Tech Collective Action LLP v. Google (opt-out) | 2024 (certified) | UK CAT | Ad-tech monopolization (collective claim) | Pending | Certified Jun 5 2024; trial listed | £13.6B claimed |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| | | | | | | Sep 2028 (12-week hearing) | |
| 33 | Kelkoo & Foundem v. Google | — | UK | Shopping-comparison damages | Pending | Claims pending | — |
| 34 | Moltiply (Trovaprezzi) v. Google | — | Italy | Shopping-comparison damages | Pending | Pending | — |
| 35 | Dutch publisher ad-tech claim (Geradin Partners) | 2022 (announced) | Amsterdam | Ad-tech damages | Pending | Pending | — |
| 36 | Epic Games v. Google (Australia) | 2021 | Federal Court of Australia | Play Store / app-store monopolization | Pending | Pending | — |
| 37 | PriceRunner (Klarna) v. Google | 2022 | Sweden Patent and Market Court | Shopping self-preferencing damages | Lost | Jul 1 2026: SEK 14.3B ≈ €1.7B incl. interest (first instance); Google weighing appeal — none confirmed filed | €1.7B |
| 38 | Idealo + Producto v. Google | — | Berlin Regional Court II | Shopping self-preferencing damages | Lost | Nov 14 2025: Idealo €374M + €91M interest; Producto €89.7M + €17.7M interest; Google planned appeal — no outcome reported | €573M |
| 39 | Twenga v. Google | — | France | Shopping self-preferencing damages | Lost | €51.5M award | €51.5M |
| 40 | European Commission: Shopping (AT.39740) | 2017 | European Commission | Self-preferencing (Art. 102 TFEU) | Lost | ECJ upheld Sep 10 2024 (final) | €2.42B |
| 41 | European Commission: Android (AT.40099) | 2018 | European Commission | Android licensing restrictions (Art. 102) | Lost | ECJ dismissed Google's final appeal Jul 2 2026 — FINAL; Alphabet jointly liable €1.52B | €4.125B |
| 42 | European Commission: AdSense (AT.40411) | 2019 | European Commission | Search-ad intermediation restrictions (Art. 102) | Won by Google | Annulled by General Court Sep 2024 | €1.49B annulled |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 43 | European Commission: Adtech (AT.40670) | 2025 | European Commission | Ad-tech self-preferencing (Art. 102) | Lost | Fine Sep 5 2025; appeal pending at General Court (17 pleas published Jan 12 2026) | €2.95B |
| 44 | European Commission DMA: Search self-preferencing (Art. 6(5)) | 2026 | European Commission (DMA) | Gatekeeper self-preferencing | Lost | Fined Jul 23 2026 (first DMA fines); Google challenging; no stay granted; compliance deadlines running | €460M |
| 45 | European Commission DMA: Play Store steering (Art. 5(4)) | 2026 | European Commission (DMA) | Gatekeeper anti-steering | Lost | Fined Jul 23 2026; Google challenging; no stay; compliance deadlines running | €430M |
| 46 | CMA: Privacy Sandbox investigation (50972) | 2021 | UK CMA | Competition concerns over cookie deprecation | Settled | Commitments accepted Feb 2022; released Oct 17 2025 | — |
| 47 | CMA: Ad tech investigation | 2022 | UK CMA | Ad-tech market power | Pending | Pending | — |
| 48 | CMA: Search SMS designation (DMCC Act) | 2025 | UK CMA | Strategic market status — market power, not wrongdoing | Pending | Designation confirmed ~Jan 2026; conduct phase pending | — |
| 49 | India CCI: search bias (Bharat Matrimony/CUTS) | 2018 | India CCI | Search bias | Lost | Rs 135.86 cr fine | ~$21M |
| 50 | India CCI: Android (MADA/AFA/RSA) | 2022 | India CCI | Android licensing | Lost | Rs 1337.76 cr; paid in full after courts declined relief — FINAL | ~$162M |
| 51 | India CCI: Play Store billing | 2022 | India CCI | Play billing policy | Lost (on appeal) | NCLAT cut fine Mar 2025 (₹936.44cr → ₹216.69cr); Supreme Court admitted cross-appeals (Google/CCI/ADIF) Aug 8 2025; hearing listed November; no verdict as of Oct 11 2026 | — |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 52 | South Korea KFTC: Android anti-fragmentation | 2021 | South Korea KFTC | Anti-fragmentation agreements | Lost | 207.4B won fine; Google appealed | ~$177M |
| 53 | South Korea KFTC: Play Store/OneStore exclusivity | 2023 | South Korea KFTC | App-store exclusivity deals | Lost | 42.1B won fine | ~$32M |
| 54 | South Korea KFTC: app-marketplace developer incentives ("Project Hug") | 2026 | South Korea KFTC | Developer incentive abuse (Jul 2019–Mar 2026; 14.16T won affected revenue) | Pending | Examiner's report Jul 1 2026; corrective measures + fine (up to 6%) recommended; commission decision pending | — |
| 55 | Japan JFTC: Android MADA/revenue-sharing | 2025 | Japan JFTC | Android licensing | Lost | Cease-and-desist Apr 2025 (no fine); 5-year monitoring | — |
| 56 | Japan JFTC: Yahoo Japan search-ad restrictions | 2024 | Japan JFTC | Search-ad distribution restrictions | Settled | Commitments; no violation finding, no fine | — |
| 57 | Australia ACCC: Telstra/Optus search pre-installation | 2025 | Australia ACCC / Federal Court | Search pre-installation deals | Lost | Federal Court order Aug 18 2025; Google admitted conduct; undertaking to remove pre-install restrictions — FINAL | A$55M (~$36M) |
| 58 | Brazil CADE: Shopping | 2013 (archived 2019) | Brazil CADE | Shopping self-preferencing | Won by Google | Archived 2019 | — |
| 59 | Brazil CADE: Android | 2019 | Brazil CADE | Android licensing | Pending | Pending | — |
| 60 | Brazil CADE: news content/AI | 2026 | Brazil CADE | News/AI content use | Pending | Formal proceedings (deepened Apr 2026) | — |
| 61 | Russia FAS: Android (Yandex complaint) | 2015 | Russia FAS | Android app bundling | Lost | Violation found; settled Apr 2017; 438M rubles paid + search-choice commitments | ~$7M |
| 62 | Turkey: Android (Yandex complaint) | 2018 | Turkey Competition Authority | Android licensing | Lost | 93M lira fine + daily 0.05% penalties | ~$15-17M |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 63 | Turkey: search-ad visibility | 2020 | Turkey Competition Authority | Search ad display | Lost | 196.6M lira fine | ~$24M |
| 64 | Turkey: local search/hotel self-preferencing (Yelp complaint) | 2021 | Turkey Competition Authority | Local search self-preferencing | Lost | 296M lira fine | — |
| 65 | Indonesia KPPU: Play billing | 2022 | Indonesia KPPU | Play billing policy | Lost | IDR 202.5B; Supreme Court rejected Google's final appeal Mar 10-13 2026 — FINAL (orders: end mandatory GPB; user-choice billing with ≥5% fee cut for one year) | ~$12.4M |
| 66 | Commissioner of Competition v. Google | 2024 | Canada Competition Tribunal | Ad-tech monopolization; seeks DFP/AdX divestiture + penalty up to 3% of global revenue | Pending | Constitutional challenge dismissed Mar 2026 | — |
| 67 | France Autorité: Google Ads suspensions | 2019 | France Autorité de la concurrence | Opaque ad-suspension rules | Lost | €150M fine | €150M |
| 68 | France Autorité: ad tech self-preferencing | 2021 | France Autorité de la concurrence | Ad-tech self-preferencing | Lost | €220M; Google didn't contest + commitments | €220M |
| 69 | France Autorité: neighboring rights (interim-measures breach) | 2021 | France Autorité de la concurrence | Press publishers' neighboring rights | Lost | €500M; appeal WITHDRAWN Jun 2022 as part of settlement (commitments binding) — FINAL | €500M |
| 70 | France Autorité: neighboring rights (commitments breach incl. AI) | 2024 | France Autorité de la concurrence | Neighboring rights / AI training data | Lost | €250M fine (Bard) | €250M |
| 71 | APIG complaint: AI Overviews / neighboring rights | 2026 | France Autorité de la concurrence | AI Overviews content use | Pending | Complaint filed Aug 2026 — newly surfaced matter | — |
| 72 | Italy AGCM: Android Auto (Enel X JuicePass) | 2021 | Italy AGCM | Android Auto interoperability (Art. 102) | Lost | €102M fine | €102M |

| # | Matter | Year | Jurisdiction | Theory | Outcome | Status as of Oct 2026 | Fine / settlement |
|---|---|---|---|---|---|---|---|
| 73 | Italy AGCM: unfair practices investigation | 2024 | Italy AGCM | Unfair commercial practices | Pending | Pending | — |
| 74 | Germany BKartA: §19a(1) paramount-significance designation | 2021 | Germany BKartA | Gatekeeper designation | Regulatory designation (no violation) | Designation concluded Dec 2021, final 2022 (no appeal); no violation finding | — |
| 75 | Germany BKartA: Google News Showcase | 2022 | Germany BKartA | Publisher terms | Settled | Commitments concluded Dec 2022 (publisher improvements) | — |
| 76 | Germany BKartA: data-processing conditions | 2023 | Germany BKartA | Data combination terms | Settled | Commitments Oct 2023 | — |
| 77 | Germany BKartA: Automotive Services/Maps Platform | 2025 | Germany BKartA | Maps/Automotive terms | Settled | Statement of objections 2023 → concluded via commitments Apr 2025 | — |

</details>

---

# Appendix C. Sources

**Google's technical report and source**

- Gemma Team, Google DeepMind. *Gemma 4 Technical Report.* arXiv:2607.02770, July 2, 2026. arxiv.org
- `google-ai-edge/LiteRT-LM`, release v0.17.1: `runtime/executor/llm_litert_npu_compiled_model_executor_utils.cc`, `…_utils_test.cc`, `runtime/executor/llm_litert_mtp_drafter.cc`, `runtime/util/CMakeLists.txt`, `runtime/core/engine_advanced.cc`, `runtime/core/CMakeLists.txt`, `runtime/core/session_advanced.cc`, `runtime/conversation/conversation.h`. Repository
- ggml `block_q4_0`. llama.cpp

**Google's issue boards**

- LiteRT-LM #3507, mapped vs. copied weights. GitHub
- LiteRT-LM PR #2076, read-only weight mapping. GitHub
- LiteRT-LM PR #1888, header read without verifier. GitHub
- TensorFlow #128148, default path mmaps without `Verify()`. GitHub
- LiteRT #10446, decoder extracted by byte offset. GitHub
- LiteRT-LM #2611, "Valid mapped region entry not found." GitHub
- LiteRT-LM #3775, bandwidth-bound kernels, reads re-split. GitHub · harness
- LiteRT-LM #3194, binary SBOM and third-party notices. GitHub
- LiteRT-LM feature request, conversation-level checkpoints and rollback (v0.15.0 / v0.17.1)
- litert-torch #1047, preserved notice. GitHub

**Google and partners**

- Google, Gemma 4 QAT, Jun 5, 2026. blog.google
- Google AI Edge, Gemma 4 at the edge, Apr 2, 2026. Google
- Google, LiteRT-LM on-device GenAI, May 19, 2026. Google
- NVIDIA, Gemma 4 on RTX, Apr 2, 2026. NVIDIA · DiffusionGemma, Jun 12, 2026. NVIDIA
- Apple Machine Learning Research, Jun 8, 2026. apple.com
- Android Developers Blog, Jul 21, 2026. googleblog.com
- Alphabet Q2 2026 CEO message. blog.google
- Google, Pixel 11 Pro XL, Aug 12, 2026. blog.google · Google Store, Tensor G6. store.google.com
- Gemma downloads: 21jingji, Aug 12, 2026 · Google, Aug 20, 2026
- Dream-RSI, Sep 2026. GitHub
- Gemini 4 Argon, Sep 30, 2026. blog.google
- EmbeddingGemma 2, Oct 6, 2026. blog.google
- Google Cloud, TPU 8t and 8i technical deep dive, Apr 22, 2026. Google Cloud
- Google Cloud pricing. cloud.google.com
- Reviewers: RunAIHome · SMF Clearinghouse · Spheron · Unsloth · kmarble.dev
- Alphabet Q2 2026 analysis. beancount.io

**Physics and measured overhead**

- Horowitz, M. "Computing's Energy Problem." *ISSCC 2014.* PDF
- Kanev, S., et al. "Profiling a Warehouse-Scale Computer." *ISCA 2015.* ACM
- Karandikar, S., et al. "A Hardware Accelerator for Protocol Buffers." *MICRO 2021.* preprint
- Langdale, G., and Lemire, D. "Parsing Gigabytes of JSON per Second." *VLDB Journal*, 2019. arXiv
- Landauer, R. 1961. doi · Bérut, A., et al. *Nature*, 2012. doi
- RFC 8259 (JSON) IETF · Protocol Buffers encoding protobuf.dev · FlatBuffers internals flatbuffers.dev · RFC 1951 (DEFLATE) RFC Editor

**Standards and law**

- NIST SP 800-145 doi · NIST SP 500-292 nist.gov · NIST SP 500-322 nist.gov · NIST SP 500-307 · NIST Handbook 44
- Apache License 2.0, §§ 3–4. apache.org
- *Jacobsen v. Katzer*, 535 F.3d 1373 (Fed. Cir. 2008). summary
- *Global-Tech Appliances, Inc. v. SEB S.A.*, 563 U.S. 754 (2011). Justia
- *Graver Tank & Mfg. Co. v. Linde Air Products Co.*, 339 U.S. 605 (1950). Justia
- *Speck v. First Evangelical Lutheran Church*, 235 S.W.3d 811 (Tex. App. 2007). summary
- *Register.com v. Verio*, 356 F.3d 393 (2d Cir. 2004). opinion
- *Oklahoma Gas & Electric Co. v. FERC*, 11 F.4th 821 (D.C. Cir. 2021) · *Entergy Louisiana, Inc. v. Louisiana PSC*, 539 U.S. 39 (2003). Justia
- 17 U.S.C. §§ 102, 506(a) Cornell LII · 18 U.S.C. § 2319 Cornell LII · 35 U.S.C. §§ 102, 154(d), 284 · 37 C.F.R. §§ 1.56, 11.18 Cornell LII
- Tex. Util. Code § 31.002(6) texas.public.law · 16 TAC § 25.5(121) Cornell LII · Texas Penal Code § 31.04 · Tex. Prop. Code chs. 12–13
- Singular Computing v. Google DCD · Sonos v. Google CNBC · IP Fray

**The GRUS record**

- Class I.I. GitHub · chloe-runtime GitHub
- RecursiveMind, live on Google Firebase. app
- Anchored Intelligence (Aug 2025) post · Digital Physics Charter (Sep 5, 2025) post · Class II and CMM (Sep 15, 2025) post
- CMM announcement, May 14, 2026. Medium
- Wikipedia revision record. Wikipedia
- *One Proposal*, with the full correspondence and the fifteen-artifact mapping. Zenodo
- *The Blunder*, technical edition, October 6, 2026.

*Nicholas Wray Cordova is the Principal and Inventor of Record of Green Recursive Utility Service LLC, Weatherford, Texas.*